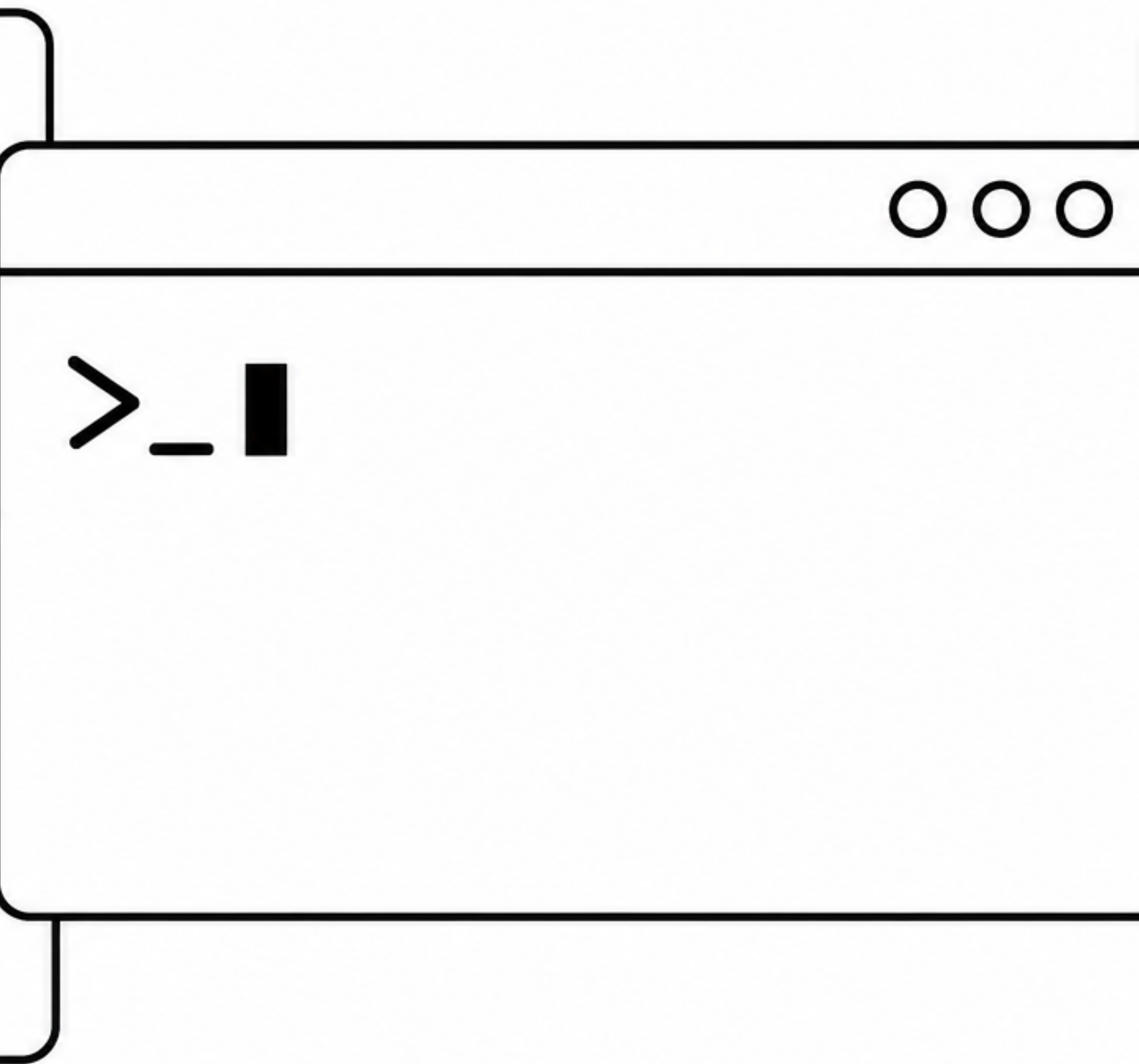




So You Want to Build Apps

A Non-Developer's Guide to Shipping
Real Software with AI

Marco Kotrotsos

April 2026

Contents

- 0 Introduction
- 1 Chapter 1: What Is a Website, Actually?
- 2 Chapter 2: Your First Page That Does Something
- 3 Chapter 3: Making It Live
- 4 Chapter 4: The Backend, Why Your App Needs a Brain
- 5 Chapter 5: Data and Databases, Where Stuff Gets Saved
- 6 Chapter 6: Building a Real App, Putting It All Together
- 7 Chapter 7: Source Control, Never Lose Your Work Again
- 8 Chapter 8: Authentication, Who Are You?
- 9 Chapter 9: Reviews, Testing, and Quality
- 10 Chapter 10: Spec-Driven Development

Introduction

You can build software now. That sentence would have been absurd five years ago if you weren't a developer. Today it's true, with caveats that matter.

AI coding tools like Claude Code, Cursor, and Codex have collapsed the barrier between "having an idea" and "having a working prototype." You can describe what you want in plain English and watch code appear on your screen. You can build a web app in an afternoon. You can deploy it to the internet before dinner.

But there's a gap between "it works on my screen" and "it works for my company." That gap is what this book is about.

Who this book is for

You're probably a product manager, a founder, an operator, or someone in a business role who has started using AI to generate code. Maybe you've built a small tool for yourself. Maybe you've prototyped something for your team. You've tasted what's possible and you want to go further, but you also know enough to worry about doing it wrong.

You don't want to become a software engineer. You want to understand enough about how software works that when an AI builds it for you, you can tell whether it built it right.

This book gives you that understanding.

What we cover

We start at the beginning. What is a website, actually? What happens when someone types a URL? What are HTML, CSS, and JavaScript, and why do you need all three? We build something small and open it in a browser.

Then we add complexity, gradually. Your page needs to do something, so we add interactivity. It needs to be on the internet, so we deploy it. It needs to remember things, so we add a database. It needs to know who's using it, so we add authentication.

Along the way, we introduce the practices that separate hobbyist code from production software: source control, code review, testing, and spec-driven development. These aren't bureaucratic overhead. They're the things that keep your software working after the first week.

By the end, you'll understand the full stack of a modern web application. Not at a developer's depth, but at a depth that lets you make good decisions, write good specs, and evaluate what your AI coding tools produce.

How to use this book

Each chapter builds on the previous one, but they're also designed to work as standalone references. If you already know what a website is, skip to Chapter 3. If you've already deployed something, jump to Chapter 5 on databases.

Every chapter includes practical examples. We use specific tools: VS Code as the editor, Claude Code as the AI assistant, Railway as the deployment platform, Clerk for authentication, and GitHub for source control. These are opinionated choices, not the only choices, but they're the ones that get you from zero to production with the least friction.

Code examples are meant to be run. If a chapter includes code, open your editor and try it. The understanding that comes from seeing something work (or break) on your own screen is different from the understanding that comes from reading about it.

A note on AI-assisted development

This book assumes you're using AI tools to write most of your code. That's fine. That's the point. But "the AI wrote it" is not the same as "I understand it." Every chapter aims to give you enough understanding that when Claude Code generates a file, you know what that file does, why it exists, and whether it's correct.

The goal is not to make you independent of AI coding tools. The goal is to make you a better operator of AI coding tools. A pilot doesn't need to build a jet engine. But a pilot who doesn't understand thrust, lift, and drag is going to have a bad time.

You're learning to be the pilot.

What you'll need

- A computer (Mac, Windows, or Linux all work)
- VS Code (free, from Microsoft)
- Claude Code or Cursor (for AI-assisted coding)
- A GitHub account (free)
- A Railway account (free tier available)
- About 2-3 hours per chapter
- Willingness to break things and fix them

No prior programming experience required. Curiosity required.

Chapter 1: What Is a Website, Actually?

This chapter strips away the mystery of how websites work. You will learn what actually happens when you type a URL into your browser, what HTML, CSS, and JavaScript are in plain terms, and why understanding these basics matters even when an AI is writing your code. By the end, you will have opened a file in your browser and understood why that counts as a website.

The Journey of a URL

You type `google.com` into your browser and hit Enter. A page appears. It feels instant, almost magical. But between your keystroke and that page appearing, a very physical process takes place, and understanding it will change how you think about every app you build.

First, your browser needs to figure out where `google.com` actually lives. Domain names like `google.com` are a convenience for humans. Computers identify each other using IP addresses, which are numerical labels like `142.250.80.46`. Your browser needs to translate the human-friendly name into the computer-friendly number.

This translation happens through something called DNS, which stands for Domain Name System. Think of it as the internet's phone book. When you type `google.com`, your browser asks a DNS server: "What's the IP address for this name?" The DNS server looks it up and responds with the number.

Your internet provider runs DNS servers, and there are also public ones run by companies like Google and Cloudflare. Your computer even keeps a small local cache of recent lookups so it doesn't have to ask every single time. The point is: before anything visual happens, there's a lookup step. Name goes in, number

comes out.

Once your browser has the IP address, it sends a request to the server at that address. A server is just a computer that's always on and always connected to the internet, sitting in a data center somewhere, waiting for requests. When it receives yours, it finds the right files and sends them back.

The request your browser sends is called an HTTP request (or HTTPS, the secure version). HTTP stands for HyperText Transfer Protocol, and it's essentially a structured way for your browser to say "please give me this page." The server receives the request, looks up the right files, and sends back an HTTP response containing those files along with a status code. You've probably seen the famous "404 Not Found" error. That's a status code telling your browser that the server couldn't find what was requested. A status of 200 means everything went fine, the file was found and sent back successfully.

What comes back is text. Specifically, it's HTML, which is a text file with special tags that tell the browser how to display content. The browser reads that HTML, follows any instructions in it (like "also go get this CSS file and this JavaScript file"), and renders the page you see on screen. A single web page often triggers dozens of additional requests: one for the CSS file, one for each JavaScript file, one for each image. The browser handles all of this automatically.

That's the whole cycle: you ask for a name, DNS translates it to an address, your browser contacts the server at that address using HTTP, the server sends back files, and your browser turns those files into a visual page.

HTML: Text With Tags

HTML stands for HyperText Markup Language, which is a mouthful that obscures how simple it actually is. HTML is plain text with annotations. Those annotations are called tags, and they tell the browser what each piece of content is.

Here's a tiny example:

```
<h1>Welcome to My Page</h1>
<p>This is a paragraph of text.</p>
```


The `<h1>` tag says "this is a heading." The `<p>` tag says "this is a paragraph." The browser reads these tags and displays the content accordingly, making the heading big and bold, giving the paragraph normal text styling.

Tags almost always come in pairs. An opening tag like `<h1>` and a closing tag like `</h1>`. Everything between them is the content that tag describes. Tags can nest inside each other too:

```
<div>
  <h1>My Title</h1>
  <p>Some text with a <strong>bold word</strong> in it.</p>
</div>
```

The `<div>` tag is a generic container, like a box that holds other elements. The `<strong>` tag makes text bold. These nest naturally: the bold word is inside the paragraph, which is inside the div.

A complete HTML file has a specific structure:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello</h1>
    <p>This is my page.</p>
  </body>
</html>
```

The `<head>` section contains information about the page (like its title, which appears in the browser tab). The `<body>` section contains everything visible on the page. That's it. Every website you've ever visited is built on this foundation.

There are a few dozen HTML tags in common use. You've seen `<h1>` for headings, `<p>` for paragraphs, `<div>` for generic containers, and `<strong>` for bold text. Others you'll encounter often include `<a>` for links, `<img>` for images, `<ul>` and `<li>` for bullet lists, `<input>` for form fields, and `<button>` for clickable buttons. You don't need to memorize them. The AI knows them all, and you'll pick up the common ones through exposure.

HTML is not a programming language. It doesn't calculate anything or make decisions. It just describes structure: this is a heading, this is a paragraph, this is a list, this is a link. Think of it as the skeleton of a web page.

CSS: Making It Look Good

If HTML is the skeleton, CSS is the skin, the clothes, and the haircut. CSS stands for Cascading Style Sheets, and it controls how HTML elements look: colors, sizes, spacing, fonts, layout.

Without CSS, a web page is just black text on a white background with default browser styling. It's readable but ugly. CSS transforms that into something that looks designed.

Here's a simple example:

```
h1 {  
  color: navy;  
  font-size: 32px;  
}  
  
p {  
  color: #333333;  
  line-height: 1.6;  
}
```

This says: make all `<h1>` elements navy-colored and 32 pixels tall. Make all `<p>` elements dark gray with generous line spacing. The syntax is straightforward: you name the element you want to style, then list the properties you want to change inside curly braces.

CSS can be written directly in the HTML file using a `<style>` tag, or it can live in a separate `.css` file that the HTML file references. In real projects, it's almost always a separate file. This separation is a design principle: HTML handles structure, CSS handles appearance.

The "cascading" part of CSS means that styles can layer on top of each other and override each other in predictable ways. If you set all paragraphs to gray but then set one specific paragraph to blue, the specific rule wins. This cascading behavior is powerful but can also be confusing, which is why CSS has a reputation for being tricky. The logic is consistent, but it takes practice to

predict how competing rules will resolve.

For your purposes as someone using AI to write code, you don't need to memorize CSS properties. There are hundreds of them. What you need to understand is the concept: CSS is the styling layer, it's separate from the content, and when something looks wrong on your page, CSS is usually where the problem lives.

JavaScript: Making It Do Things

HTML gives you structure. CSS gives you appearance. JavaScript gives you behavior.

JavaScript is the programming language of the web browser. When you click a button and a menu appears, that's JavaScript. When you type in a search box and suggestions appear below it, that's JavaScript. When a form checks your email format before you submit it, that's JavaScript.

Unlike HTML and CSS, JavaScript is a real programming language. It can make decisions ("if the user clicked this button, show that panel"), store information ("remember that the user selected 'dark mode'"), and manipulate the page ("add a new row to this table").

Here's a tiny example:

```
document.getElementById("myButton").addEventListener("click", function() {  
    document.getElementById("message").textContent = "You clicked the button!";  
});
```

Even without understanding every word, you can probably read the intent: find the element called "myButton," and when someone clicks it, change the text of the element called "message" to say "You clicked the button!"

JavaScript runs in the browser, on the user's computer. This is an important distinction. When you load a web page, the HTML, CSS, and JavaScript files all download to your machine, and the JavaScript executes right there in your browser. It's not running on some remote server (though we'll talk about server-side code later in the book). It's running on your device.

This is what makes websites feel interactive. The page doesn't need to reload every time something changes. JavaScript can modify what you see in real time, responding to your clicks, your typing, your scrolling.

Your First "Website"

You can create a website right now, on your computer, without installing anything, without an internet connection, without any special tools.

Open any text editor. On a Mac, TextEdit works (switch to plain text mode first). On Windows, Notepad. On either, something like VS Code is better, but not required. Type this:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First Page</title>
  </head>
  <body>
    <h1>I made this</h1>
    <p>It's not much, but it's mine.</p>
  </body>
</html>
```

Save the file as `my-page.html`. The `.html` extension is important. Now find that file in your file explorer and double-click it. Your browser will open it and display your heading and paragraph.

That's a website. A real one. The browser doesn't care whether the HTML file came from a server in Virginia or from your desktop. It reads the HTML and renders it the same way.

Try changing the text, saving the file, and refreshing the browser. You'll see your changes immediately. This feedback loop, edit, save, refresh, is the fundamental cycle of web development. Professionals use fancier tools that automate parts of this cycle, but the core loop is the same.

Now add some style. Update your file:

```

<!DOCTYPE html>
<html>
  <head>
    <title>My First Page</title>
    <style>
      body {
        font-family: Arial, sans-serif;
        max-width: 600px;
        margin: 0 auto;
        padding: 40px;
        background-color: #f5f5f5;
      }
      h1 {
        color: #2c3e50;
      }
      p {
        color: #555;
        line-height: 1.6;
      }
    </style>
  </head>
  <body>
    <h1>I made this</h1>
    <p>It's not much, but it's mine.</p>
  </body>
</html>

```

Save and refresh. Now you have a centered, styled page with a pleasant color scheme. Still just a file on your computer, still just text with tags, but it looks intentional now.

Notice what happened here. You went from a plain, unstyled page to a designed-looking one by adding about ten lines of CSS. The HTML content didn't change at all. The structure is the same, only the presentation shifted. This separation between content and presentation is one of the most powerful ideas in web development, and it's why you can completely redesign a website without touching the underlying content.

If you want to experiment further, try adding a link. Put this inside your `<body>`:

```

<a href="https://example.com">Click here to visit Example.com</a>

```

The `<a>` tag creates a clickable link. The `href` attribute tells the browser where to go when someone clicks it. Refresh the page and you'll see the link. Click it and your browser navigates to that URL. You've just connected your page to the rest of the internet.

File on Your Computer vs. File on a Server

The page you just made works perfectly on your machine. But if you email the link to a friend, they'll see nothing. The path `file:///Users/you/Desktop/my-page.html` points to a location on your computer. Nobody else can access it.

For other people to see your page, the HTML file needs to live on a server, a computer that's publicly accessible on the internet with an IP address that DNS can point to. When you "deploy" a website (which we'll cover in Chapter 3), you're essentially copying your files to one of these servers.

That's the entire difference between a local file and a live website: where the file lives. The HTML is identical. The browser renders it the same way. The only question is whether other people's browsers can reach it.

This is worth internalizing because it demystifies a lot of web development terminology. "Hosting" means renting space on a server. "Deploying" means putting your files on that server. "Going live" means your files are now on a server that DNS can route to. None of these are complicated concepts. They're just logistics of putting files where others can find them.

Why This Matters for AI-Assisted Development

You might be wondering why any of this matters when Claude Code or Cursor can write the HTML, CSS, and JavaScript for you. Fair question. The answer is that understanding what these technologies are, even at a surface level, makes you dramatically better at directing the AI.

When you tell an AI "make me a website," you'll get something generic. When you tell it "create an HTML page with a navigation bar that uses flexbox layout, a

main content area with 600px max-width, and a footer that sticks to the bottom of the viewport," you'll get something specific and useful. You don't need to know how to write flexbox CSS from memory. You just need to know that flexbox is a CSS layout system and roughly what it does.

When something goes wrong (and it will), you need to know where to look. If the page structure is wrong, the problem is in the HTML. If it looks wrong, the problem is in the CSS. If it doesn't respond to clicks or inputs, the problem is in the JavaScript. This basic triage ability saves enormous amounts of time.

You also need to recognize when the AI generates something reasonable versus something bizarre. AI coding tools occasionally produce HTML structures that technically work but are semantically wrong, like using a table for page layout or nesting divs fifteen levels deep for no reason. If you don't know what HTML is supposed to look like, you won't catch these issues until they cause real problems.

Think of it like driving a car with an automatic transmission. You don't need to understand the physics of internal combustion. But you do need to know that the car runs on fuel, that the steering wheel controls direction, and that weird grinding noises mean something is wrong. This chapter gave you that level of understanding for web technologies.

What You Now Know

After reading this chapter, you can:

- **Explain what happens when someone visits a website:** DNS translates the domain name to an IP address, the browser contacts the server at that address, the server sends back HTML/CSS/JS files, and the browser renders them into a visual page.
- **Distinguish HTML, CSS, and JavaScript by their roles:** HTML defines structure and content, CSS controls appearance and layout, JavaScript adds interactivity and behavior.
- **Create and view a basic HTML page on your own computer:** Write HTML in a text editor, save it as a `.html` file, and open it in a browser to see the result.

- **Understand the difference between a local file and a deployed website:**

The HTML is identical; the only difference is whether the file lives on your machine or on a publicly accessible server.

- **Triage problems in AI-generated web code:** Structure issues point to HTML, visual issues point to CSS, and behavior issues point to JavaScript.

Chapter 2: Your First Page That Does Something

In Chapter 1, you learned what websites are made of and opened a static HTML file in your browser. This chapter moves past static. You will make buttons respond to clicks, forms process input, and pages change without reloading. You will build a working expense calculator by prompting Claude Code, learn the edit-save-refresh feedback loop that every developer relies on, and understand the common mistakes that trip people up when asking AI to write interactive code.

Beyond Static Pages

The page you built in Chapter 1 was static. It displayed text, it looked nice, but it just sat there. You could read it, but you couldn't interact with it. Most useful software needs to respond to what users do: click a button, fill out a form, select an option, drag an item.

The difference between a static page and an interactive one is JavaScript. CSS can do some light interaction (hover effects, simple animations), but anything that involves logic, calculations, or dynamic content changes requires JavaScript.

Consider what happens when you use a mortgage calculator on a bank's website. You type in a loan amount, an interest rate, and a term length. You click "Calculate." The monthly payment appears. No page reload, no waiting for a server, just instant results. That's JavaScript reading the values you typed, running a formula, and displaying the answer.

Or think about a to-do list app. You type a task, press Enter, and it appears in a list below. You click a checkbox to mark it done. You click a delete button to

remove it. Every one of those interactions is JavaScript listening for events (typing, clicking) and modifying the page in response.

The good news: you don't need to write this JavaScript yourself. That's what AI coding tools are for. But you need to understand what JavaScript does so you can ask for the right things and recognize whether the AI gave you something that works.

JavaScript for Non-Programmers

JavaScript is a set of instructions that the browser follows. That's the simplest accurate description. When you write JavaScript (or have an AI write it for you), you're telling the browser: "When this happens, do that."

Three concepts cover about 80% of what you need to understand:

Variables store information. A variable is a named container. You might have a variable called `totalExpenses` that holds the number `1450.75`. You can change what's in the container, read from it, or use it in calculations. It's like a labeled box where you put things.

```
let totalExpenses = 0;
let userName = "Marco";
let isLoggedIn = true;
```

Variables can hold numbers, text (called "strings" in programming), or true/false values (called "booleans"). They can also hold more complex things, but those three cover most everyday cases.

Functions are reusable blocks of instructions. Instead of writing the same steps over and over, you package them into a function and give it a name. Then you can run those steps whenever you need them by calling the function.

```
function calculateTotal(price, quantity) {
  return price * quantity;
}
```

This function takes a price and a quantity, multiplies them, and gives back the result. You can use it anywhere in your code by writing

`calculateTotal(29.99, 3)`, which would return `89.97`.

Events are things that happen. A click is an event. A keystroke is an event. The page finishing loading is an event. JavaScript lets you attach functions to events: "When the user clicks this button, run this function."

```
button.addEventListener("click", function() {  
    // Do something when the button is clicked  
});
```

That's the core of interactive web pages. Variables hold data. Functions process data. Events trigger functions. Everything else is details.

You will never need to memorize JavaScript syntax. The AI handles that. What you need is the mental model: interactive features work because JavaScript listens for events, reads and stores data in variables, and runs functions that change what's on the page.

Building an Expense Calculator

Enough theory. You're going to build something real: a simple expense calculator that lets you add items with descriptions and amounts, displays a running total, and lets you delete items. This is a genuinely useful tool, and building it will teach you how the AI-assisted development cycle works.

If you're using Claude Code, open your terminal and navigate to a folder where you want to work. You might create a new folder:

```
mkdir expense-calculator  
cd expense-calculator
```

Give Claude Code a clear prompt. Not "make me an expense calculator." That's too vague. Instead:

Create a single HTML file called `index.html` with an expense calculator. It should have:

- An input field for expense description
- An input field for expense amount (numbers only)
- An "Add" button that adds the expense to a list below
- Each item in the list shows the description, the amount formatted as currency, and a "Delete" button
- A running total at the bottom that updates when items are added or deleted
- Clean, minimal styling with a centered layout, max-width 500px
- All CSS and JavaScript should be inline in the same HTML file

Claude Code will generate a complete file. Open `index.html` in your browser. You should see a form with two input fields, a button, and an empty list area with a total of \$0.00.

Type "Lunch" in the description field and "14.50" in the amount field. Click Add. The item should appear in the list, and the total should update to \$14.50. Add a few more items. Delete one. The total should adjust.

If it works, congratulations. You just built functional software without writing a single line of code yourself.

If something doesn't work, that's normal too. Welcome to development.

The Feedback Loop

The most important habit in building software is the tight feedback loop: make a change, see the result, adjust. Professional developers do this hundreds of times per day. The cycle looks like this:

1. Change the code (or ask the AI to change it)
2. Save the file
3. Switch to your browser and refresh the page
4. Check if it works the way you expected
5. If not, figure out what's wrong and go back to step 1

For a single HTML file like your expense calculator, this loop is beautifully simple. You edit the file, save it, switch to your browser, press `Ctrl+R` (or

Cmd+R on Mac), and see the result. The whole cycle takes seconds.

When you're working with Claude Code, the loop has an extra participant. Instead of editing the code directly, you describe what you want to change:

```
The delete button is too close to the amount text. Add some left margin
to the delete button. Also, make the total bold and slightly larger
than the list items.
```

Claude Code makes the changes. You refresh the browser. You see the result. You either accept it or ask for another adjustment.

This loop is where AI-assisted development really shines. You can iterate on visual design, behavior, and layout by describing what you want in plain English. Each cycle takes a few seconds. In traditional development, you'd need to look up CSS properties, figure out the right selector, and write the code yourself. With AI assistance, you just describe the desired outcome.

But the loop only works if you actually check the result every time. Don't ask for five changes at once and then check. Ask for one, check it, then ask for the next. Small iterations catch problems early. Large batches of changes create confusion when something breaks and you don't know which change caused it.

Reading the Code (Yes, You Should)

You don't need to write code, but you should learn to read it at a basic level. When Claude Code generates your expense calculator, open the HTML file in a text editor and scroll through it. You don't need to understand every line, but you should be able to identify the major sections.

Look for the `<style>` tag. Everything between `<style>` and `</style>` is CSS. This is where the visual appearance is defined.

Look for the `<body>` tag. Everything between `<body>` and `</body>` is the page structure. You should see your input fields, your button, and a container for the expense list.

Look for the `<script>` tag. Everything between `<script>` and `</script>` is JavaScript. This is where the behavior lives: what happens when you click Add, how the total gets calculated, how the delete button works.

Try to match what you see in the code with what you see in the browser. The input field in the HTML corresponds to the text box on the page. The CSS rules for that input control how it looks. The JavaScript function that handles the Add button determines what happens when you click it.

This reading ability becomes critical when things go wrong. If the calculator displays amounts without dollar signs, you can look at the JavaScript and search for where the amount gets displayed. You might see something like:

```
amountSpan.textContent = expense.amount;
```

And realize it should probably be:

```
amountSpan.textContent = "$" + expense.amount.toFixed(2);
```

You don't need to write that fix yourself. You can tell the AI: "The amounts aren't showing dollar signs or consistent decimal places. Fix the formatting to show currency format like \$14.50." But recognizing where the problem lives, in the display logic rather than in the calculation or the HTML structure, helps you communicate with the AI more effectively.

Making It More Useful

Your basic expense calculator works, but it has limitations. This is where iterative prompting comes in. Each prompt should address one specific improvement:

Adding categories:

```
Add a dropdown select field for expense categories: Food, Transport, Entertainment, Bills, Other. Show the category next to each expense item in the list. Add a small colored dot before each category to make them visually distinct.
```

Persistence between sessions:

Save the expenses to `localStorage` so they persist when I close and reopen the browser. Load any saved expenses when the page opens.

Summary statistics:

Add a summary section below the total that shows: the number of expenses, the highest single expense, and a breakdown of spending per category.

Each of these is a meaningful addition to your tool. After each one, refresh the browser and test it. Add some expenses, check that categories appear, close the browser and reopen it to test persistence, verify the summary numbers are correct.

This incremental approach, start simple, add features one at a time, test after each addition, is how professional software gets built. The only difference is that instead of writing code, you're writing prompts. The engineering discipline is the same.

Common Mistakes When Prompting AI for UI Code

After working with dozens of people who are new to AI-assisted development, certain patterns of prompting mistakes come up repeatedly. Knowing these will save you frustration.

Being too vague about layout. "Make it look nice" gives the AI no useful information. What does "nice" mean? Centered or full-width? Light or dark? Compact or spacious? Instead, reference specific things: "Center the form with a max-width of 400px, add 20px padding, use a light gray background with white cards for each expense item, and use the system default sans-serif font."

Describing what it should look like instead of what it should do. "I want a box that shows numbers" is less useful than "I want a running total that updates automatically when items are added or removed from the list." The AI needs to understand the behavior, not just the appearance. What triggers the update? What data feeds into it? What format should the output be?

Asking for everything at once. "Build me a full expense tracker with categories, charts, export to CSV, multi-currency support, recurring expenses, and budget alerts" is a recipe for a mess. The AI will produce a large, complex file that's hard to test and hard to debug. Start with the basic version and add features incrementally. Each addition builds on something you've already tested and verified.

Not specifying edge cases. What should happen if someone enters a negative amount? What about zero? What if they leave the description blank? What if they type letters in the amount field? These edge cases are where bugs hide. The AI might handle them, or it might not. Include your expectations: "Don't allow empty descriptions. Don't allow negative or zero amounts. Show an error message below the input if the user tries to submit invalid data."

Forgetting to specify the file structure. Should the AI create one HTML file or separate HTML, CSS, and JavaScript files? For simple tools, a single file is easier to manage. For anything more complex, separate files are better organized. Tell the AI which approach you want: "Keep everything in a single HTML file" or "Create separate files for HTML, CSS, and JavaScript."

Ignoring mobile. If there's any chance someone will open your page on a phone, say so: "Make the layout responsive so it works on mobile screens. Stack the input fields vertically on screens narrower than 500px." The AI won't add responsive design unless you ask for it.

What the Browser Developer Tools Can Tell You

Every modern browser has built-in developer tools. On Chrome, press F12 or Ctrl+Shift+I (Cmd+Option+I on Mac). On Firefox and Edge, the same shortcuts work.

You don't need to master these tools, but two tabs are immediately useful:

The Elements tab shows the HTML structure of the page. You can hover over elements to see them highlighted on the page. You can click on an element to see its CSS styles on the right side. This is invaluable when something looks wrong: click the element that looks off, check its styles, and you'll often spot the issue immediately.

The Console tab shows JavaScript errors. If your expense calculator's Add button stops working, open the Console and look for red error messages. These messages are often cryptic, but you can paste them directly into your AI prompt: "When I click the Add button, nothing happens. The console shows this error: 'TypeError: Cannot read properties of null (reading addEventListener)'. Fix this."

That error message, by the way, usually means the JavaScript tried to find an element on the page but couldn't, often because the element's ID in the HTML doesn't match the ID the JavaScript is looking for. Knowing that won't help you fix it manually, but it helps you give the AI better context.

The developer tools are your diagnostic dashboard. When something works, you don't need them. When something breaks, they tell you where and why. Get comfortable opening them and looking at the Console tab whenever something isn't behaving as expected.

From Calculator to Application

The expense calculator you built in this chapter is a single HTML file. It runs entirely in the browser. It stores data in `localStorage`, which means the data lives on your device and disappears if you clear your browser data.

This is fine for personal tools. A surprising number of useful applications can be built this way: calculators, converters, planning tools, checklists, simple dashboards. If the tool is just for you, a single HTML file with `localStorage` is genuinely sufficient.

But there are clear limits. Other people can't access your tool unless you send them the file. Data doesn't sync between devices. There's no user accounts, no login, no way to share data between users. There's no server-side processing, so you can't send emails or connect to external APIs without exposing credentials.

In Chapter 3, you'll take the first step past these limits by deploying your work to the internet. And in later chapters, you'll add server-side code, databases, and the other pieces that turn a single-page tool into a real application.

For now, the important thing is that you've built something functional. You've experienced the feedback loop. You've seen how to prompt an AI for interactive code and how to iteratively improve the result. These skills transfer directly to

everything else in this book.

What You Now Know

After reading this chapter, you can:

- **Explain what JavaScript does:** It listens for user events (clicks, keystrokes), processes data using variables and functions, and updates the page without reloading.
- **Build a working tool by prompting an AI coding assistant:** Write clear, specific prompts that describe behavior, not just appearance, and iterate one feature at a time.
- **Use the edit-save-refresh feedback loop:** Make changes, save the file, refresh the browser, and verify the result before moving on to the next change.
- **Use browser developer tools for basic debugging:** Open the Console tab to find JavaScript errors, and use the Elements tab to inspect HTML structure and CSS styles.
- **Avoid common prompting mistakes:** Be specific about layout, describe behavior rather than appearance, handle edge cases explicitly, and add features incrementally rather than all at once.

Chapter 3: Making It Live

You have a working expense calculator sitting on your computer. It runs in your browser, it does useful things, and nobody else can see it. This chapter fixes that. You will learn what deployment actually means, how domain names work, and how to put your project on the internet using Railway. By the end, you will have a real URL that anyone in the world can visit.

What "Deploying" Means

Deployment is a word that sounds more technical than it is. Deploying your application means copying your files to a computer that's connected to the internet and configured to serve those files to anyone who asks for them.

That's it. When a developer says "I deployed the app," they mean they moved their code from their laptop to a server. When a company says "we're deploying a new version," they mean they're replacing the old files on the server with new ones.

The server itself is usually not a physical box that someone owns. Most of the time, it's a virtual server, a slice of computing power rented from a company like AWS, Google Cloud, or Railway. These companies operate massive data centers full of servers and let you use a piece of one for a monthly fee. Some platforms, like Railway, abstract this even further so you never think about servers at all. You just push your code and they handle the rest.

The reason deployment feels intimidating to newcomers is that it involves several steps that are each simple but unfamiliar: setting up an account on a hosting platform, connecting your code, configuring a few settings, and triggering the build. Each step takes minutes once you know what to do, but the first time through, you're learning a new interface while also trying to understand the concepts. That's what this chapter is for.

Domain Names and DNS, Revisited

In Chapter 1, you learned that DNS translates domain names into IP addresses. Now you need to understand this from the other direction, because you'll want your deployed application to have a domain name.

When you deploy to a platform like Railway, you'll get a free URL that looks something like `your-app-production.up.railway.app`. This works perfectly fine. Anyone with that URL can visit your application. But it's long, it's not memorable, and it has "railway" in it, which looks unpolished if you're sharing the link professionally.

If you want a custom domain like `expenses.yourdomain.com`, you need two things: ownership of the domain and a DNS record that points it to your server.

Buying a domain is straightforward. Registrars like Namecheap, Cloudflare, or Google Domains sell domain names. A `.com` domain costs roughly \$10-15 per year. You search for the name you want, buy it, and now you control the DNS records for that domain.

DNS records are entries in the internet's phone book that you control for your domain. The most common type is an A record, which points a domain name to an IP address. Another common type is a CNAME record, which points a domain name to another domain name (like pointing `expenses.yourdomain.com` to `your-app-production.up.railway.app`).

When you set up a custom domain with Railway, the process is:

1. Tell Railway you want to use `expenses.yourdomain.com`
2. Railway gives you a CNAME value to point to
3. Go to your domain registrar and create a CNAME record pointing your subdomain to Railway's value
4. Wait a few minutes (sometimes up to 48 hours, but usually much less) for the DNS change to propagate
5. Railway automatically handles the SSL certificate so your site uses HTTPS

You don't need a custom domain to get started. The free Railway URL is fine for testing, for personal projects, and even for sharing with colleagues. Custom domains are a polish step, not a prerequisite.

Version Control: A Brief Detour

Before you deploy, you need to understand one concept that sits between "code on your laptop" and "code on a server": version control, specifically Git and GitHub.

Git is a tool that tracks changes to your files over time. Every time you save a meaningful change (called a "commit"), Git records what changed and when. If you break something, you can go back to a previous version. If you want to try something risky, you can create a branch, a parallel version of your code, and experiment without affecting the working version.

GitHub is a website that hosts Git repositories (a repository is a project folder tracked by Git). When you push your code to GitHub, you're uploading your files and their entire change history to a remote server. This serves as a backup, a collaboration tool, and, crucially for us, a bridge to deployment.

Most deployment platforms, including Railway, can connect directly to your GitHub repository. When you push new code to GitHub, Railway detects the change and automatically redeploys your application. This is called continuous deployment, and it's the standard way modern applications get updated.

Setting up Git for the first time involves a few commands:

```
git init
git add .
git commit -m "Initial version of expense calculator"
```

Then, after creating a repository on GitHub:

```
git remote add origin https://github.com/yourusername/expense-calculator.git
git push -u origin main
```

If this looks intimidating, Claude Code can walk you through each step. The AI handles Git commands well and can explain what each one does. The key concepts to remember: `commit` saves a snapshot of your changes locally, `push` uploads those snapshots to GitHub, and your deployment platform watches GitHub for new pushes.

Railway: From Zero to Deployed

Railway is a deployment platform that optimizes for simplicity. It's not the only option (Vercel, Render, and Fly.io are alternatives), but it strikes a good balance between ease of use and capability for someone just getting started.

Here's the complete process for deploying your expense calculator.

Step 1: Create a Railway Account

Go to railway.com and sign up. The easiest path is to sign up with your GitHub account, because Railway needs GitHub access anyway to deploy your code. Railway offers a free tier that's sufficient for personal projects and learning.

Step 2: Prepare Your Project

Your expense calculator is a single HTML file. To deploy it as a website, you need a minimal server to serve that file. Claude Code can set this up for you:

```
I have a single index.html file that I want to deploy on Railway.
Create a minimal Node.js server that serves this file. I need:
- A package.json with the dependencies
- A server.js file that serves the index.html on port 3000
  (or whatever PORT environment variable Railway provides)
- Keep it as simple as possible
```

The AI will create two files:

package.json, which lists your project's dependencies (in this case, just Express, a popular web server library for Node.js):

```
{
  "name": "expense-calculator",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

server.js, which is a small program that serves your HTML file:

```
const express = require("express");
const path = require("path");
const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.static(path.join(__dirname, "public")));

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Move your `index.html` file into a folder called `public`. Your project structure should look like:

```
expense-calculator/
  package.json
  server.js
  public/
    index.html
```

Step 3: Test Locally

Before deploying, verify it works on your machine. In your terminal:

```
npm install
npm start
```

Open `http://localhost:3000` in your browser. You should see your expense calculator. If it works locally, it will almost certainly work when deployed.

Step 4: Push to GitHub

If you haven't already, initialize Git and push your code:

```
git init
git add .
git commit -m "Initial version with Express server"
git remote add origin https://github.com/yourusername/expense-calculator.git
git push -u origin main
```

Step 5: Deploy on Railway

1. Log into Railway and click "New Project"
2. Select "Deploy from GitHub repo"
3. Find and select your expense-calculator repository
4. Railway detects the Node.js project and starts building automatically
5. Wait about a minute for the build to complete
6. Click "Generate Domain" to get a public URL

That's it. Your expense calculator is now live on the internet. Copy the URL and open it in a different browser, or send it to someone. They'll see your application running.

Step 6: Make Updates

The beauty of the GitHub-Railway connection is that updating your live application is now trivial:

1. Make changes to your code locally (or have Claude Code make them)
2. Test locally by refreshing `localhost:3000`
3. Commit and push to GitHub:

```
git add .
git commit -m "Added category feature"
git push
```

4. Railway automatically detects the push and redeploys

Within a minute or two, your live URL reflects the changes. No manual deployment steps, no uploading files, no restarting servers. Push to GitHub and it just happens.

From Localhost to a Real URL

The moment your application gets a public URL is a psychological milestone. The thing you built is no longer a file on your computer. It's software that exists on the internet. People can use it. You can put it in an email signature, share it on LinkedIn, or hand the link to your boss.

But there are a few things to understand about this transition.

Your application now runs on someone else's computer. Railway's servers are running your Node.js code 24/7, serving your HTML file to anyone who requests it. If Railway's servers go down, your site goes down. This is true for every website on the internet, they all depend on their hosting provider's reliability.

The free tier has limits. Railway's free plan gives you a certain amount of compute hours and bandwidth per month. For a simple HTML tool like your expense calculator, you'll never hit these limits. But if you eventually build something that gets thousands of daily users or processes large files, you'll need a paid plan.

HTTPS is handled for you. Railway automatically provides an SSL certificate, which means your site uses `https://` instead of `http://`. This is important because browsers mark non-HTTPS sites as "not secure," which looks bad and can scare users away. With Railway, you don't have to think about this.

Your data still lives in the browser. Remember, your expense calculator stores data in `localStorage`, which is specific to each user's browser. If someone visits your URL and adds expenses, those expenses are stored in their browser, not on your server. This is fine for a personal tool. If you want shared data (multiple users seeing the same information), you'll need a database, which is a topic for a later chapter.

The Gap Between "Works on My Machine" and "Works for Everyone"

Every developer, from absolute beginners to senior engineers, encounters this gap. Your application works perfectly on your laptop but fails in some way when deployed. Understanding why this happens helps you anticipate and avoid common issues.

Environment differences. Your laptop runs a specific version of Node.js, a specific operating system, and has specific files and folders. The server might have different versions or a different OS. Most deployment platforms, Railway included, let you specify which versions you need, but you have to know to do this.

Missing environment variables. Some applications need configuration values: API keys, database URLs, secret tokens. These are typically stored as environment variables, not in the code itself (because putting secrets in code is a security risk). If your code expects an environment variable that exists on your laptop but not on the server, it will fail. Railway has a simple interface for setting environment variables for your deployed application.

File path issues. Code that references `C:\Users\You\project\data.json` will fail on a Linux server where that path doesn't exist. Use relative paths in your code, not absolute ones. If your AI writes code with absolute paths, ask it to fix them.

Network and CORS issues. If your application tries to call external APIs (like a weather service or a stock price API), you might run into CORS restrictions, which are browser security rules about which websites can call which APIs. This doesn't come up with your expense calculator, but it will come up later. When it does, the error message will mention "CORS" or "cross-origin," and that's your cue to tell the AI: "I'm getting a CORS error when calling this API. Can you set up the request to go through my server instead?"

Case sensitivity. Your Mac or Windows laptop might not care about the difference between `Image.png` and `image.png`. A Linux server (which is what most hosting platforms run) absolutely does. If your HTML references `image.png` but the file is named `Image.png`, it will work on your laptop and break in production. Always use consistent, lowercase file names.

These issues sound like a lot, but most of them are one-time lessons. Once you've been bitten by a case-sensitivity issue once, you'll lowercase your filenames forever. The point isn't to memorize a list of potential problems. The point is to know that "works on my machine" is a starting point, not a finish line.

First Deployment Checklist

Before you deploy any project, run through this checklist. It takes two minutes and prevents the most common failures:

Does it work locally? Run your application on your own machine and test the key features. If it doesn't work locally, it won't work deployed.

Are file names lowercase and consistent? Check for any uppercase letters in file names and references to those files in your code. Rename anything that's inconsistent.

Are there any hardcoded paths? Search your code for file paths that include your username, your home directory, or drive letters like `C:\`. These need to be relative paths.

Are secrets out of the code? API keys, passwords, and tokens should be in environment variables, not hardcoded in your JavaScript. If you're deploying to Railway, add them in the Railway dashboard under Variables.

Is the start command defined? Your `package.json` needs a `start` script that tells the server how to run your application. Without this, Railway won't know how to start your code.

Does the app use the PORT environment variable? Railway assigns a port number dynamically. Your server code should read `process.env.PORT` instead of hardcoding a port number. The AI usually gets this right, but check.

Is a .gitignore file present? This file tells Git which files to skip when committing. At minimum, it should include `node_modules/` (because those get installed on the server from your `package.json`) and any `.env` files (because those contain secrets). Ask Claude Code: "Create a `.gitignore` file for a Node.js project."

If everything passes, push to GitHub and deploy. If something fails after deployment, Railway's logs (visible in the dashboard) will tell you what went

wrong. Copy the error message and share it with your AI assistant. Nine times out of ten, it can diagnose and fix the issue from the log output alone.

What Comes Next

Your expense calculator is now live. You built something from nothing, made it interactive, and put it on the internet. That progression, from blank file to deployed application, is the same progression every piece of software goes through. The tools and complexity scale up, but the fundamental steps don't change.

In the chapters ahead, you'll face the questions that naturally follow. How do you store data permanently? How do you let users log in? How do you connect to external services? How do you build something that multiple people can use simultaneously?

Each of those questions has answers that follow the same pattern you've already learned: understand the concept, tell the AI what you want, check the result, iterate. The technology gets more sophisticated, but the process stays the same.

What You Now Know

After reading this chapter, you can:

- **Explain what deployment means in concrete terms:** Copying your application files to a server that's publicly accessible on the internet, so anyone with the URL can use your application.
- **Understand how domain names, DNS records, and HTTPS work together:** Domain registrars sell names, DNS records point names to servers, and SSL certificates (handled automatically by Railway) make the connection secure.
- **Deploy a project to Railway via GitHub:** Create a GitHub repository, connect it to Railway, generate a domain, and set up automatic redeployment on every push.
- **Anticipate common deployment issues:** Environment differences, case sensitivity, hardcoded paths, missing environment variables, and CORS errors

are the usual suspects when something works locally but fails in production.

- **Run a pre-deployment checklist:** Verify local functionality, check file naming consistency, remove hardcoded paths, externalize secrets, confirm the start command, and ensure the PORT variable is used.

Chapter 4: The Backend, Why Your App Needs a Brain

So far, everything you've built runs entirely in the browser. HTML renders the structure, CSS makes it look good, JavaScript makes it interactive. That's genuinely useful for many things. But at some point, you'll hit a wall. You'll want to save data that persists after the user closes the tab. You'll want to send emails, process payments, or talk to another service's API. You'll want to keep an API key secret instead of shipping it inside your JavaScript where anyone can read it. That's when you need a backend, a separate program running on a server that handles the work your browser can't or shouldn't do. This chapter explains what a backend is, when you actually need one, and how to build a simple one using Express.js with the help of Claude Code.

What Is a Server?

The word "server" sounds intimidating, but the concept is simple. A server is a computer that listens for requests and sends back responses. That's it. When you visit a website, your browser sends a request to a server somewhere, and that server responds with HTML, CSS, JavaScript, images, or data. The server is just another computer, often sitting in a data center, running a program that knows how to handle those requests.

Your laptop can be a server. When you run a program locally during development, your machine is both the client (your browser making requests) and the server (the program responding to them). That's what "localhost" means. You're talking to yourself.

In production, the server is someone else's computer. When you deployed to Railway in Chapter 3, Railway gave you a machine in a data center that runs your program and responds to requests from real users. But the principle is identical to what happens on your laptop.

Here's the key mental model: a backend is a program that sits between your frontend (the browser) and everything else. It's the middleman. The browser asks the backend for data, and the backend goes and gets it, whether from a database, another API, or its own logic.

Why You Need a Backend

There are exactly three reasons you need a backend, and most apps hit at least two of them.

Reason 1: Saving data. When a user fills out a form and clicks submit, where does that data go? Without a backend, nowhere useful. JavaScript in the browser can store small amounts of data in `localStorage`, but that's tied to one browser on one device. If the user clears their browser data, it's gone. If they switch to their phone, it's gone. A backend stores data in a database that persists regardless of what the user does on their end.

Reason 2: Talking to other services. Your app probably needs to send emails, charge credit cards, check the weather, or pull data from a CRM. These services provide APIs (we'll cover what that means in a minute), but many of them require authentication, meaning you need an API key or secret token to use them. You cannot put those secrets in your frontend JavaScript. Anyone who opens the browser's developer tools can see your entire JavaScript source code. That means they'd see your API keys and could use them to send emails from your account, charge your credit card, or access your CRM data.

Reason 3: Running logic you don't want exposed. Maybe you have a pricing algorithm, a scoring system, or business rules that you don't want competitors to see. Frontend code is public. Backend code is private. Only the results get sent to the browser, never the logic itself.

If your project doesn't need any of these three things, you might not need a backend at all. A portfolio website, a landing page, a calculator that runs entirely in the browser: these are all fine as static HTML/CSS/JS. Don't add complexity for the sake of it.

API Endpoints: URLs That Return Data Instead of Pages

When you type a URL into your browser, you get back a webpage. But not all URLs return webpages. Some return raw data, usually in a format called JSON. These are API endpoints.

An API endpoint is just a URL that your code can call to get or send data. Instead of returning pretty HTML with styling and images, it returns structured data that your program can work with.

Here's the difference:

- Visit `https://example.com/about` in your browser, and you get a rendered webpage with headings, paragraphs, and images.
- Your code calls `https://api.example.com/users/42` and gets back something like `{"id": 42, "name": "Sarah", "email": "sarah@example.com"}`.

That second response isn't meant for humans to read. It's meant for programs to consume. Your frontend JavaScript calls the API endpoint, gets the data back, and then decides how to display it on the page.

When you build a backend, a big part of the work is creating these endpoints. You're defining URLs that your frontend (or other services) can call to get data, create records, update information, or delete things.

The four main operations map to HTTP methods:

- **GET**: Retrieve data. "Give me the list of customers."
- **POST**: Create something new. "Add this new customer."
- **PUT**: Update something existing. "Change this customer's email."
- **DELETE**: Remove something. "Delete this customer."

These four operations cover about 90% of what any API does. If you understand them, you understand the backbone of web development.

Building a Simple API with Express.js

Express is a Node.js framework for building backends. It's been around since 2010, it's minimal, it's well-documented, and nearly every Node.js tutorial uses it. That last point matters, because it means Claude Code and other AI tools have seen millions of Express examples in their training data. They're very good at writing Express code.

Here's what a minimal Express server looks like:

```

const express = require('express');
const app = express();

app.use(express.json());

let tasks = [
  { id: 1, title: 'Buy groceries', done: false },
  { id: 2, title: 'Write report', done: true }
];

// GET all tasks
app.get('/api/tasks', (req, res) => {
  res.json(tasks);
});

// GET a single task
app.get('/api/tasks/:id', (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.json(task);
});

// POST a new task
app.post('/api/tasks', (req, res) => {
  const task = {
    id: tasks.length + 1,
    title: req.body.title,
    done: false
  };
  tasks.push(task);
  res.status(201).json(task);
});

// DELETE a task
app.delete('/api/tasks/:id', (req, res) => {
  tasks = tasks.filter(t => t.id !== parseInt(req.params.id));
  res.status(204).send();
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});

```

Don't worry about memorizing this. The point is to understand the pattern. Each `app.get()`, `app.post()`, and `app.delete()` defines an endpoint. When someone (usually your frontend code) sends a request to that URL, the function

runs and sends back a response.

A few things to notice:

The data is stored in a variable. That `tasks` array lives in the server's memory. If you restart the server, the data is gone. In Chapter 5, we'll replace this with a real database. But for learning the pattern, an in-memory array is fine.

`req` **and** `res` **are the request and response.** Every endpoint handler receives these two objects. `req` contains everything the client sent (URL parameters, request body, headers). `res` is how you send something back.

Status codes communicate outcomes. 200 means success. 201 means "created successfully." 404 means "not found." 204 means "done, nothing to send back." These numbers aren't arbitrary. They're an agreed-upon language between client and server. You don't need to memorize them all. When Claude Code writes your API, it'll pick the right ones. But knowing that 404 means "not found" and 500 means "the server broke" will help you debug problems.

The `:id` syntax means "this part of the URL is a variable." When someone calls `/api/tasks/7`, the number 7 becomes available as `req.params.id`. This is how you build endpoints that work with specific records.

Running Your First API

To run this, you need Node.js installed and Express added to your project. If you followed along in earlier chapters, you already have Node.js. Adding Express is one command:

```
npm install express
```

Save the code above in a file called `server.js`, then run it:

```
node server.js
```

Your server is now running on port 3000. You can test it by opening `http://localhost:3000/api/tasks` in your browser, and you'll see the JSON response with your two tasks.

For testing POST and DELETE requests, the browser address bar won't work, since it only sends GET requests. You can use a tool like Thunder Client (a VS Code extension) or just ask Claude Code to write a quick test script that calls your endpoints.

When You Need a Backend vs When Static HTML Is Enough

This is a decision you'll face with every project, and getting it right saves you time. Here's a simple framework:

You don't need a backend when:

- The content doesn't change based on who's viewing it (marketing site, documentation, portfolio)
- All interactions happen within the browser (calculator, converter, single-player game)
- You're building a prototype that doesn't need to save anything yet
- The page just displays static information

You need a backend when:

- Users create accounts or log in
- Data needs to persist between sessions (saved settings, user content, transactions)
- You're connecting to third-party services that require API keys
- Different users see different data
- You need to send emails, process files, or run scheduled tasks
- You have business logic that should stay private

There's a gray area. Some static site generators can handle forms by sending data to third-party services like Formspree or Netlify Forms. You can use services like Firebase or Supabase that provide a backend-as-a-service, so you write frontend code that talks directly to their APIs. These can be valid shortcuts for simple use cases, but as your app grows, having your own backend gives

you full control.

The honest advice: if you're unsure whether you need a backend, you probably will eventually. But start without one if you can. Add it when the first real need appears. Building a backend before you need one is a classic over-engineering mistake.

How to Ask Claude Code to Build an API for You

This is where things get practical. Claude Code is excellent at building Express APIs, but the quality of what it builds depends entirely on how you ask.

Bad prompt:

"Build me an API"

This gives Claude Code nothing to work with. What data? What operations? What rules?

Better prompt:

"Build an Express.js API for managing a task list. I need endpoints to list all tasks, get a single task by ID, create a new task, update a task's status, and delete a task. Each task has a title (string, required), a description (string, optional), a status (pending, in-progress, or done, defaults to pending), and a created timestamp. Store tasks in memory for now. Add basic input validation, return proper HTTP status codes, and include error handling."

See the difference? The second prompt tells Claude Code:

- **What framework to use** (Express.js)
- **What the data looks like** (title, description, status, timestamp)
- **What operations are needed** (list, get, create, update, delete)
- **What rules apply** (title is required, status has specific allowed values, defaults)
- **What quality you expect** (validation, status codes, error handling)

You can go even further. Add constraints like "use RESTful URL patterns," "add CORS headers so the frontend can call from a different port," or "group related routes using Express Router." The more specific you are, the better the result.

Here's a prompt pattern that works well for APIs:

"Build an Express.js API for [domain]. The data model is: [list fields with types and constraints]. I need these endpoints: [list operations]. Business rules: [list any logic]. Include input validation, proper error responses, and CORS support. Store data in [memory / SQLite / PostgreSQL]."

After Claude Code generates the API, test every endpoint. Send a request with missing required fields and make sure you get a proper error response. Try to create invalid data and verify it's rejected. This is where bugs hide in AI-generated code: not in the happy path, but in the edge cases.

The Request-Response Cycle

Before we move on, there's one concept worth understanding clearly, because it'll help everything else click. Every interaction between your frontend and backend follows the same cycle:

1. The frontend sends a **request** (GET /api/tasks)
2. The server receives the request
3. The server does whatever work is needed (fetch from database, validate input, run logic)
4. The server sends a **response** (JSON data, a success message, or an error)
5. The frontend receives the response and updates the UI

This cycle happens every time. When you click "save" in an app, the frontend sends a POST request to the backend with the data, the backend saves it, and the backend responds with confirmation. When a page loads and shows your data, the frontend sent a GET request, the backend fetched your data from the database, and sent it back.

Understanding this cycle is more important than memorizing any specific framework or syntax. Frameworks change. The request-response cycle has been how the web works since the beginning, and it's not going anywhere.

A Note on Security

Once you have a backend, you have responsibilities. Your server is accessible from the internet, which means anyone can send requests to it. Without proper precautions, someone could send a DELETE request to wipe your data, or send malformed input to crash your server.

The good news: basic security is well-understood and Claude Code can implement it. When you ask for an API, include "add input validation and sanitization" in your prompt. When you add authentication (Chapter 8), your endpoints will verify that the person making the request is who they claim to be.

For now, keep two principles in mind:

- 1. Never trust input from the client.** Always validate and sanitize data on the server side, even if you also validate on the frontend. Frontend validation is a courtesy to the user. Backend validation is what actually protects your data.
- 2. Never put secrets in your frontend code.** API keys, database passwords, encryption keys: these all live on the server, stored in environment variables (which we'll cover in Chapter 6).

What You Now Know

You now understand why apps need a backend: to save data, communicate with other services, and keep secrets and logic private. You know that a server is just a computer running a program that listens for requests and sends responses. You know what API endpoints are, that they're URLs that return data instead of web pages, and that the four HTTP methods (GET, POST, PUT, DELETE) cover most of what any API needs to do. You've seen what Express.js code looks like and how each endpoint maps to a specific URL and operation. You know when a backend is necessary versus when static HTML is enough. And you know how to prompt Claude Code to build an API by being specific about your data model, operations, and business rules. In the next chapter, we'll solve the problem of data disappearing when the server restarts by adding a real database.

Chapter 5: Data and Databases, Where Stuff Gets Saved

In the last chapter, we stored data in a JavaScript array inside the server's memory. That works for learning, but it has an obvious problem: restart the server and everything is gone. Every deployment, every crash, every server update wipes the slate clean. Real applications need data that survives restarts, that multiple users can read and write at the same time, and that can be searched and filtered efficiently. That's what databases do. This chapter covers the two databases you'll encounter most as a new builder, SQLite and PostgreSQL, explains when to use each one, and shows you how to work with them through AI prompts instead of writing SQL by hand.

Why You Can't Just Save to a File

Your first instinct might be reasonable: why not just write the data to a JSON file on disk? When the server starts, read the file. When data changes, write it back. Simple.

It works for a personal project with one user. But it falls apart fast, for three specific reasons.

Concurrency. What happens when two users submit data at the exact same time? User A reads the file, adds their record, and writes it back. But User B also read the file, before User A's write finished. User B adds their record and writes the file. User A's data is now gone, overwritten by User B's version. This is called a race condition, and it's one of the most common bugs in software. Databases handle this by design. They manage concurrent reads and writes so

your data stays consistent even when hundreds of users are hitting the server simultaneously.

Structure and querying. A JSON file is a blob of data. Want to find all orders from last Tuesday? You'd have to read the entire file into memory, parse it, and loop through every record checking dates. With a database, that's a single query that runs in milliseconds, even with millions of records. Databases are built to search, filter, sort, and aggregate data efficiently.

Scale. A JSON file with 100 records is fine. A JSON file with 100,000 records takes seconds to read and parse every time someone makes a request. A database handles millions of records without breaking a sweat because it uses indexes, which are basically pre-built lookup tables that let it find data without scanning everything.

The lesson: files work for configuration and static content. For anything users create, modify, or interact with, use a database.

The Spreadsheet Analogy

If you've used Excel or Google Sheets, you already understand 80% of how relational databases work. The concepts map almost directly.

A **database** is like a workbook, a single file that contains multiple sheets.

A **table** is like a sheet. Each table stores one type of thing: users, orders, products, tasks. You'd never dump customers and invoices on the same sheet in Excel, and you wouldn't put them in the same database table either.

Columns define the structure. Just like a spreadsheet has column headers (Name, Email, Date Joined), a database table has columns with specific types. A column might hold text, numbers, dates, or true/false values. Unlike a spreadsheet, a database enforces these types. If you defined a column as a number, it won't let you put "hello" in it. This strictness prevents bad data from getting in.

Rows are the individual records. Each row is one customer, one order, one task. When you add a new customer, you're adding a row.

A **primary key** is a unique identifier for each row. In a spreadsheet, you might have a row number. In a database, you typically have an `id` column that

auto-increments: 1, 2, 3, 4. This guarantees every record can be uniquely identified.

A **query** is like a filter in Excel. "Show me all customers in Texas" or "Show me orders over \$500 from last month" are things you might filter for in a spreadsheet. In a database, you write these as SQL queries. But, and this is the important part, you don't have to write them yourself. That's what Claude Code is for.

SQLite: The Simplest Database

SQLite is a database that lives in a single file. No installation. No server to run. No configuration. Your app opens a file, reads and writes data to it, and that's the whole story.

This simplicity makes SQLite perfect for starting out. Here's why:

- **Zero setup.** No service to install or manage. The database is just a file called something like `database.sqlite` sitting in your project folder.
- **No credentials.** No username, password, or connection string to configure. Your code just points to the file.
- **Self-contained.** You can copy the file, email it to someone, or back it up by dragging it to another folder.
- **More capable than you'd expect.** SQLite handles thousands of concurrent reads and millions of rows. It's used in production by apps you use every day. Every iPhone and Android device uses SQLite. Firefox and Chrome use SQLite. It's the most deployed database in the world.

Here's what using SQLite looks like in a Node.js project:

```

const Database = require('better-sqlite3');
const db = new Database('tasks.sqlite');

// Create the table if it doesn't exist
db.exec(`
  CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    status TEXT DEFAULT 'pending',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
  )
`);

// Add a task
const insert = db.prepare(
  'INSERT INTO tasks (title, description) VALUES (?, ?)'
);
insert.run('Buy groceries', 'Milk, eggs, bread');

// Get all tasks
const tasks = db.prepare('SELECT * FROM tasks').all();
console.log(tasks);

```

You'll notice the SQL statements inside the strings: `CREATE TABLE`, `INSERT INTO`, `SELECT * FROM`. SQL (Structured Query Language) is the standard language for talking to relational databases. It reads almost like English. "Select everything from the tasks table." "Insert into the tasks table these values." You don't need to become fluent in SQL. You need to understand what it does at a high level, and then let Claude Code write the specific statements.

PostgreSQL: The Production Database

SQLite is great for getting started, but it has one significant limitation: it doesn't handle multiple simultaneous writers well. If your app has many users creating and updating data at the same time, SQLite can become a bottleneck because it locks the entire database during writes.

PostgreSQL (often called Postgres) solves this. It's a database that runs as a separate service, handling connections from your app and managing concurrent reads and writes efficiently. It's the database behind Instagram, Spotify, and thousands of other production applications.

The tradeoff is complexity. Postgres needs to be installed, configured, and run as a service. It requires a username, password, and connection string. You have to manage backups. You have to monitor it.

Or, you can let someone else handle all of that.

Railway's Database Add-On

Remember Railway from Chapter 3? In addition to hosting your app, Railway can host a PostgreSQL database with about three clicks. No installation, no configuration, no maintenance on your part.

Here's the process:

1. Open your Railway project
2. Click "New Service" and select "Database"
3. Choose PostgreSQL
4. Railway provisions the database and gives you a connection string

That connection string looks something like this:

```
postgresql://user:password@host:port/database
```

It's a single URL that contains everything your app needs to connect: the username, password, server address, port, and database name. You store this as an environment variable (more on those in Chapter 6) and your code uses it to connect.

Using Postgres with Node.js looks like this:

```

const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL
});

// Get all tasks
async function getTasks() {
  const result = await pool.query(
    'SELECT * FROM tasks ORDER BY created_at DESC'
  );
  return result.rows;
}

// Add a task
async function addTask(title, description) {
  const result = await pool.query(
    'INSERT INTO tasks (title, description) VALUES ($1, $2) RETURNING *',
    [title, description]
  );
  return result.rows[0];
}

```

The SQL is nearly identical to SQLite. The main difference is how you connect. Instead of pointing to a file, you're connecting to a service over the network using that connection string. The queries themselves are the same language.

Writing Queries Through AI: Prompt Patterns That Work

Here's the honest truth about SQL: it's not hard to learn, but you don't need to learn it right now. Claude Code can write every query you need, as long as you tell it what you want clearly.

The key is describing your data and your intent, not the SQL syntax. Here are prompt patterns that produce good results:

For creating tables:

"Create a SQLite table called 'customers' with these columns: id (auto-incrementing primary key), name (required text), email (required text, unique), company (optional text), created_at (timestamp, defaults to now), and

active (boolean, defaults to true)."

Notice how you're specifying constraints: required vs optional, unique, defaults. These constraints are your business rules encoded in the database structure, and they prevent bad data from ever getting stored.

For queries:

"Write a query that gets all active customers who signed up in the last 30 days, sorted by name."

"Write a query that counts the number of orders per customer, only including customers with more than 5 orders."

"Write a query that updates the status to 'shipped' for all orders placed before yesterday that are still marked as 'pending'."

For relationships between tables:

"I have a customers table and an orders table. Each order belongs to one customer. Add a customer_id foreign key to the orders table. Then write a query that gets each customer's name along with their total order amount."

For the complete setup:

"Set up SQLite in my Express.js project using better-sqlite3. Create tables for a simple CRM: customers (name, email, company, phone) and interactions (linked to customer, type of interaction, notes, date). Create API endpoints to add customers, list customers with search, add an interaction to a customer, and get a customer with all their interactions. Include proper error handling."

That last prompt gives Claude Code everything it needs: the database library, the data model, the relationships, the endpoints, and the quality expectations. You'll get a working API with a real database in one prompt.

A warning about AI-generated queries. Always review queries that modify or delete data. A `DELETE FROM` task without a `WHERE` clause deletes every task in the table. AI tools usually get this right, but "usually" is not "always." Before running any destructive query, read it and make sure the `WHERE` clause exists and makes sense.

Choosing Between SQLite and PostgreSQL

This decision is simpler than most guides make it seem. Here's a practical framework:

Start with SQLite when:

- You're building something for the first time and figuring out your data model
- Your app has a small number of users (under a few hundred concurrent)
- You want the simplest possible setup with no external dependencies
- You're building a prototype or internal tool
- You want to be able to copy your entire database as a single file

Switch to PostgreSQL when:

- You're deploying to production with real users
- You have many concurrent users writing data
- You need advanced features like full-text search, JSON columns, or complex queries
- You're deploying to Railway or another cloud platform (Postgres is the default there)
- You need your database to be separate from your app server (for scaling or reliability)

The migration path. Because both SQLite and Postgres use SQL, moving from one to the other is manageable. The table structures are nearly identical. The queries are 95% the same. There are small syntax differences, but Claude Code can handle the translation. Don't agonize over this choice. Pick SQLite to start. When you hit its limits, switch to Postgres. You'll know when it's time.

A common pattern for experienced developers: use SQLite during local development because it's simple and fast, and Postgres in production because it handles scale. Your code checks an environment variable to decide which database to connect to. This gives you the best of both worlds.

What a Schema Looks Like in Practice

Let's look at a slightly more realistic database design. Say you're building an internal tool that tracks client projects. You might have three tables:

```
CREATE TABLE clients (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  email TEXT UNIQUE,  
  company TEXT,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE projects (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  client_id INTEGER NOT NULL,  
  name TEXT NOT NULL,  
  status TEXT DEFAULT 'active',  
  budget REAL,  
  start_date DATE,  
  FOREIGN KEY (client_id) REFERENCES clients(id)  
);  
  
CREATE TABLE time_entries (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  project_id INTEGER NOT NULL,  
  description TEXT,  
  hours REAL NOT NULL,  
  date DATE NOT NULL,  
  FOREIGN KEY (project_id) REFERENCES projects(id)  
);
```

Three tables, connected by relationships. Each project belongs to a client (via `client_id`). Each time entry belongs to a project (via `project_id`). This is called a relational database because the tables relate to each other through these foreign keys.

The power of this structure: you can ask questions across tables. "Show me total hours billed per client this month." "Which projects are over budget?" "List all active projects for a specific client." The database can answer these by joining the related tables together.

You don't need to design a schema like this from scratch. Describe your use case to Claude Code, and tell it to design the database schema. It'll produce

something reasonable. Review it, adjust the column names and types to match your domain language, and iterate from there.

Database Mistakes to Avoid

A few common mistakes that new builders make with databases:

Not backing up. For SQLite, this means copying the database file regularly. For Postgres on Railway, Railway handles backups, but verify this is enabled. Losing your database is losing your app's memory.

Storing passwords in plain text. Never do this. When you add authentication (Chapter 8), the auth service handles password hashing for you. If you're ever tempted to create a password column in your users table, stop and use a proper auth service instead.

Not adding indexes. If you frequently search by a specific column (like email or date), add an index on that column. An index is like the index in the back of a book: it lets the database find records without scanning every page. Ask Claude Code to "add an index on the email column of the customers table" and it'll write the one-line SQL statement.

Making columns required when they shouldn't be. Think carefully about which columns are NOT NULL (required). If you make too many fields required, users will get frustrated filling out forms. If you make too few required, you'll get incomplete data. The balance depends on your specific use case.

Not having a created_at timestamp. Always add a created_at column with a default of the current time. You'll be surprised how often you need to know when a record was created, and you can't add this information retroactively.

What You Now Know

You now understand why files aren't sufficient for storing application data: concurrency issues, lack of structure, and poor performance at scale. You know the spreadsheet analogy for relational databases, where tables are sheets, columns are headers, and rows are records. You've seen SQLite, a database that lives in a single file with zero setup, and PostgreSQL, the production-grade database that handles concurrent users and complex queries. You know that

Railway can provision a Postgres database in clicks, giving you a connection string your app uses to communicate. You understand how to prompt Claude Code to write SQL queries by describing your data and intent in plain English. And you have a clear framework for choosing between SQLite and Postgres: start simple, upgrade when you need to. In the next chapter, we'll combine everything, frontend, backend, and database, into a single structured project using Next.js.

Chapter 6: Building a Real App, Putting It All Together

You now have all the pieces. You know what frontend and backend code do. You know how APIs work. You know how databases store data. The question is: how do you combine all of this into one coherent project that actually works? This chapter walks through building a real internal tool from start to finish. We'll organize a project properly, use Next.js to combine frontend and backend in a single codebase, handle secrets safely with environment variables, and write the kind of specification that makes Claude Code produce good results on the first try. By the end, you'll have built something you could deploy on Monday and hand to your team.

From Scattered Files to a Structured Project

In the earlier chapters, we worked with individual files. An `index.html` here, a `server.js` there, maybe a `database.sqlite` somewhere. That works for learning, but a real application has dozens or hundreds of files, and without organization, you'll lose track of what's where within a week.

Every serious project follows a folder structure. Not because developers enjoy bureaucracy, but because a predictable structure means anyone (including you, six months later) can open the project and immediately know where to find things.

Here's a minimal but effective structure for a web application:

```
my-app/  
  src/  
    app/          -- pages and routes  
    components/   -- reusable UI pieces  
    lib/          -- shared utilities and database code  
  public/         -- static files (images, fonts)  
  .env.local      -- secrets (never committed to Git)  
  package.json    -- project dependencies  
  README.md       -- what this project is and how to run it
```

A few principles behind this:

Pages go in one place, components in another. A page is a full screen the user sees. A component is a reusable piece like a button, a form, or a navigation bar. Separating them means you can find any page by looking in one folder and any reusable piece in another.

Utilities and database code go in lib. This is shared code that multiple pages or API routes might use. Database connection setup, helper functions, validation logic, it all lives here so you don't duplicate it across files.

Static files go in public. Images, downloadable PDFs, favicon, anything that gets served as-is without processing. Put it in `public` and it's accessible by URL.

Secrets go in .env.local. This file never gets uploaded to GitHub. It contains your database passwords, API keys, and other values that should stay private. We'll cover this in detail later in this chapter.

Why Next.js? Because It Does Everything

Up until now, we've treated the frontend and backend as separate things. Your HTML/CSS/JS runs in the browser. Your Express server runs on a separate process. They communicate through API calls. This works, and plenty of production apps are built this way.

But it's also more to manage. Two codebases, two deployments, two sets of configuration. For a team of developers, this separation can make sense. For a solo builder or small team, it's overhead you don't need.

Next.js combines both. It's a React framework that gives you:

- **Pages** that render in the browser (frontend)
- **API routes** that run on the server (backend)
- **Server components** that can talk to your database directly
- **Built-in routing** based on your folder structure
- **One deployment** that handles everything

When you create a file at `src/app/dashboard/page.js`, that becomes the `/dashboard` page on your website. When you create a file at `src/app/api/tasks/route.js`, that becomes the `/api/tasks` endpoint. The folder structure is the routing. No configuration needed.

This convention-over-configuration approach is what makes Next.js particularly good for AI-assisted development. When you tell Claude Code to "add a settings page," it knows exactly where to put the file and what the URL will be. There's no ambiguity.

React, the library underneath Next.js, is a way of building user interfaces from components. Instead of writing one massive HTML file, you break your interface into pieces: a header component, a sidebar component, a task list component, a task card component. Each piece manages its own display and behavior. You compose them together like building blocks.

If you've never used React, don't worry about learning it separately. When you ask Claude Code to build pages, it'll write React components for you. What matters is understanding the organizational principle: pages are made of components, components can be reused across pages, and data flows from top to bottom.

Building a Real Internal Tool: Step by Step

Theory is helpful. Building something is better. Let's walk through building a client project tracker, the kind of internal tool that a consulting firm, agency, or freelancer might use to track active projects, log time, and see which clients are generating the most revenue.

Step 1: Create the Project

Next.js has an official project generator. One command creates the entire project structure:

```
npx create-next-app@latest project-tracker
```

It'll ask a few questions. Accept the defaults for now, except make sure you select "Yes" for the App Router and "Yes" for Tailwind CSS. The App Router is the modern way Next.js handles routing, and Tailwind CSS is a utility-first CSS framework that makes styling much faster when working with AI tools.

Step 2: Define Your Data Model

Before writing any code, write down what data your app needs to manage. For a client project tracker:

Clients: name, email, company, phone, created date **Projects:** name, client (linked), status (active/completed/on-hold), budget, start date, end date **Time entries:** project (linked), description, hours, date, billable (yes/no)

This is your schema. It's the foundation everything else builds on. Get this right and the rest flows naturally. Get it wrong and you'll be restructuring your database later. Spend ten minutes thinking about what fields you actually need before you start building.

Step 3: Set Up the Database

For this project, we'll use SQLite during development and can switch to Postgres for production later. Ask Claude Code:

"Set up SQLite in this Next.js project using better-sqlite3. Create tables for clients, projects, and time_entries based on this schema: [paste your data model]. Create a database utility file at src/lib/db.js that initializes the database and exports query functions for each table. Include functions for CRUD operations on all three tables, plus a function that gets a project with total hours logged."

Claude Code will create the database file, the utility module, and all the query functions. Review what it produces, check that the column types and constraints

match your intentions, and adjust anything that looks off.

Step 4: Build the API Routes

Next.js API routes live in the `src/app/api/` directory. Each route is a file that exports functions for different HTTP methods. Ask Claude Code:

"Create API routes for the project tracker. I need routes for: `/api/clients` (GET list, POST create), `/api/clients/[id]` (GET single, PUT update, DELETE), `/api/projects` (GET list with optional client filter, POST create), `/api/projects/[id]` (GET with total hours, PUT update, DELETE), `/api/time-entries` (POST create, GET list with project filter), `/api/time-entries/[id]` (DELETE). Use the database functions from `src/lib/db.js`. Add input validation for required fields. Return proper HTTP status codes and error messages."

This gives you a complete API. Every endpoint maps to a URL, accepts requests, calls the database functions, and returns responses. Test each endpoint using a tool like Thunder Client or by asking Claude Code to write a test script.

Step 5: Build the Pages

Now the frontend. You need a few pages:

- **Dashboard** (`/`): Overview showing active project count, total hours this week, recent activity
- **Clients list** (`/clients`): Table of all clients with search
- **Client detail** (`/clients/[id]`): Single client with their projects
- **Projects list** (`/projects`): All projects with status filter
- **Project detail** (`/projects/[id]`): Single project with time entries and total hours
- **Time entry form** (`/projects/[id]/log-time`): Form to log hours against a project

For each page, give Claude Code a prompt like:

"Create the clients list page at `src/app/clients/page.js`. It should fetch clients from `/api/clients` and display them in a table with columns for name, company,

email, and date added. Add a search input that filters the table as the user types. Each row should link to the client detail page. Include an 'Add Client' button that opens a modal with a form. Use Tailwind CSS for styling. Keep the design clean and functional, not decorative."

Notice the specifics: what data to show, how it's organized, what interactions exist, and what the styling approach should be. "Clean and functional, not decorative" is the kind of direction that keeps AI tools from generating elaborate designs when you just need a working tool.

Step 6: Connect the Pieces

At this point, you have pages that call API routes that query a database. The full stack in one project. When a user opens `/clients`, the page component makes a fetch call to `/api/clients`, which calls the database function, which queries the SQLite file, which returns the data up through the chain to the user's screen.

Test the full flow. Add a client through the form. Does it appear in the list? Click into the client detail. Are their projects showing? Log time against a project. Does the total update? These end-to-end tests are where you'll find integration bugs, places where the frontend sends data in a format the backend doesn't expect, or where the API returns data shaped differently from what the frontend assumes.

Environment Variables: Keeping Secrets Out of Your Code

Every real app has values that should never appear in your source code. Database connection strings, API keys for email services, secret tokens for authentication. These are environment variables.

In Next.js, you store them in a file called `.env.local` at the root of your project:

```
DATABASE_URL=postgresql://user:password@host:5432/mydb
EMAIL_API_KEY=sk-abc123def456
APP_SECRET=some-random-string-nobody-should-know
```

In your code, you access them with `process.env`:


```
const dbUrl = process.env.DATABASE_URL;
```

Three important rules:

Never commit `.env.local` to Git. Next.js adds this to `.gitignore` by default, but double-check. If you accidentally push your secrets to GitHub, anyone who can see your repository can see your database password. Rotate your credentials immediately if this happens.

Create a `.env.example` file that shows the structure without real values.

This helps anyone setting up the project (including future you) know which environment variables are needed:

```
DATABASE_URL=postgresql://user:password@host:5432/mydb
EMAIL_API_KEY=your-api-key-here
APP_SECRET=generate-a-random-string
```

This file gets committed to Git because it contains no real secrets.

Set environment variables in your hosting platform. Railway has a "Variables" tab in each service where you paste your production values. These are encrypted and never exposed in your code or Git history. When your app runs on Railway, `process.env.DATABASE_URL` contains the production value you set there, not the development value from your local `.env.local` file.

Next.js has a specific rule about environment variables in the browser: only variables prefixed with `NEXT_PUBLIC_` are available in frontend code. Everything else is server-only. This is a safety net. If your API key is called `EMAIL_API_KEY` (no prefix), it's impossible for it to leak to the browser, even by accident. If you name it `NEXT_PUBLIC_EMAIL_API_KEY`, it will be bundled into the JavaScript sent to every user. Don't do that with secrets.

The Full Stack in One Project

Take a step back and look at what you've built:

```

project-tracker/
  src/
    app/
      page.js                -- Dashboard
      clients/
        page.js              -- Clients list
        [id]/
          page.js             -- Client detail
      projects/
        page.js              -- Projects list
        [id]/
          page.js             -- Project detail
          log-time/
            page.js           -- Time entry form
    api/
      clients/
        route.js             -- Clients API
        [id]/
          route.js            -- Single client API
      projects/
        route.js             -- Projects API
        [id]/
          route.js            -- Single project API
      time-entries/
        route.js             -- Time entries API
        [id]/
          route.js            -- Single time entry API
    components/
      ClientForm.js          -- Reusable client form
      ProjectCard.js         -- Project display card
      TimeEntryRow.js        -- Time entry table row
      SearchInput.js         -- Search component
    lib/
      db.js                  -- Database setup and queries
  public/
    logo.png
  .env.local                 -- Secrets
  .env.example               -- Secret template
  package.json

```

Pages and API routes live in `app/`. Reusable pieces live in `components/`. Database logic lives in `lib/`. Secrets live in `.env.local`. That's the full stack. Frontend, backend, database, and configuration, all in one project, one Git repository, one deployment.

How to Spec This for Claude Code

The single biggest factor in whether Claude Code builds something good or something you'll throw away is the quality of your specification. Here's how to write one that works.

Start with a project brief. Before writing any code, create a file called `SPEC.md` or `DESIGN.md` at the root of your project:

```
# Project Tracker

## Purpose
Internal tool for tracking client projects and time entries.
Used by a team of 5 consultants.

## Data Model
- Clients: name (required), email (unique), company, phone
- Projects: name (required), linked to client, status
  (active/completed/on-hold), budget, start/end dates
- Time entries: linked to project, description, hours (required),
  date (required), billable flag

## Pages
- Dashboard: active projects count, hours this week, recent activity
- Clients list: searchable table, add client modal
- Client detail: info + linked projects
- Projects list: filterable by status
- Project detail: info + time entries + total hours
- Log time: form to add time entry to a project

## Tech Stack
- Next.js with App Router
- SQLite via better-sqlite3
- Tailwind CSS for styling

## Rules
- All forms validate required fields before submitting
- Deleting a client should warn about linked projects
- Time entries display in reverse chronological order
- Dashboard shows data from the current week
```

Now when you work with Claude Code, you can reference this spec:

"Read the SPEC.md file. Build the database setup and all API routes first. Then build the dashboard page."

You can also put this specification in your project's `CLAUDE.md` file, which Claude Code reads automatically at the start of every session. That way, every conversation starts with Claude Code understanding your project's purpose, data model, and conventions.

Break work into chunks. Don't ask Claude Code to build the entire app in one prompt. Work through it systematically:

1. Database setup and query functions
2. API routes for each resource
3. Dashboard page
4. Clients pages
5. Projects pages
6. Time entry pages
7. Polish and edge cases

Each chunk is testable on its own. You can verify the API routes work before building the pages that use them. This incremental approach catches problems early, when they're cheap to fix.

Be specific about behavior, not implementation. Tell Claude Code what should happen, not how to code it:

- Good: "When the user deletes a client who has active projects, show a confirmation dialog listing the project names and asking if they want to proceed."
- Less good: "Use a React modal component with `useEffect` to check project count and conditionally render a dialog with `Array.map` over project names."

The first prompt describes the behavior you want. Claude Code will figure out the implementation. The second prompt micromanages the code, which constrains Claude Code and sometimes leads to worse solutions than it would have chosen on its own.

Include edge cases. AI tools are very good at the happy path, where everything goes right. They're less reliable with the sad path, where things go wrong. Explicitly mention edge cases in your prompts:

- "What happens if the user submits the form with empty required fields?"
- "What if the database is unreachable?"
- "What if two users edit the same project simultaneously?"
- "What if the user navigates to a client ID that doesn't exist?"

Calling out these scenarios pushes Claude Code to handle them, instead of hoping they never happen.

A Realistic Workflow

Here's what a typical building session looks like once you have the basics down:

1. **Open your project in VS Code with Claude Code active**
2. **Review your spec** (or update it based on what you learned)
3. **Pick one feature to build** (one page, one API route, one component)
4. **Write a detailed prompt** describing the feature
5. **Review the generated code** (read through it, does the logic make sense?)
6. **Test it** (does it actually work when you click around?)
7. **Fix issues** (tell Claude Code what's broken, be specific about the symptom)
8. **Commit your work** (save your progress in Git)
9. **Pick the next feature**

This cycle takes about 15-30 minutes per feature once you're comfortable. A full internal tool with 5-6 pages and a database can be built in a day or two. Not because the work is trivial, but because Claude Code handles the boilerplate, the repetitive code, and the syntax details. Your job is architecture, decisions, and testing.

The part that takes the longest isn't writing code. It's deciding what to build: what the data model should be, what pages you need, what happens when things go wrong. Those decisions are yours, and they're the most valuable part of the process.

What You Now Know

You now understand how to structure a real project with folders for pages, components, utilities, and static files. You know why Next.js is a good choice for full-stack applications, combining frontend and backend in a single codebase with file-based routing. You've walked through building a complete internal tool from database schema to finished pages, step by step. You know how environment variables keep your secrets safe and why `.env.local` should never be committed to Git. You've seen what a good specification looks like and how to break work into chunks that Claude Code can handle effectively. And you know the practical workflow: spec, build, test, fix, commit, repeat. In the next chapter, we'll cover Git and GitHub, the system that makes sure you never lose your work and can always roll back when something breaks.

Chapter 7: Source Control, Never Lose Your Work Again

Every professional who has ever lost a document to a crashed hard drive, an accidental overwrite, or a bad edit knows the sinking feeling. Source control eliminates that feeling permanently. Git, the tool developers use for tracking changes, records every edit you make, lets you undo anything, and makes collaboration possible without stepping on each other's work. You don't need to become a Git expert. But if you're building software, even with AI doing most of the coding, understanding source control is the difference between a hobby project and something you can actually maintain and grow.

The Problem Git Solves

Think about how you handle documents at work. You probably have files named things like `proposal_v2_final.docx`, `proposal_v2_final_ACTUAL.docx`, and `proposal_v2_final_Marco_edits_USE_THIS_ONE.docx`. Everyone has been there. It's a mess, and eventually somebody opens the wrong version and overwrites hours of work.

Now imagine that problem multiplied by hundreds of files that all depend on each other. That's a software project. Change one file and three others might break. Make an edit on Tuesday and realize on Friday it was wrong. Your colleague makes changes to the same file at the same time you do.

Git solves all of this. It's a system that tracks every change to every file in your project, who made the change, when they made it, and why. Think of it as an infinite undo button combined with a time machine. You can go back to any

point in your project's history and see exactly what the code looked like. You can try something risky, and if it doesn't work, revert to where you were before with a single command.

Git was created in 2005 by Linus Torvalds, the same person who created Linux. He built it because the existing source control tools were too slow for a project as large as the Linux kernel. Today, virtually every software project in the world uses Git. It's not optional. It's infrastructure.

GitHub: Where Your Code Lives Online

Git runs on your computer. It tracks changes locally. But what happens if your laptop dies? What if you want to work from a different machine? What if you're collaborating with someone else?

That's where GitHub comes in. GitHub is a website that hosts Git repositories (a repository, or "repo," is just a project folder that Git is tracking). Think of it as Google Drive for code, except it's specifically designed for software development. Your code lives on GitHub's servers, backed up, versioned, and accessible from anywhere.

GitHub adds features on top of Git: a web interface for browsing code, tools for code review, issue tracking, project management, and a lot more. When someone says "the code is on GitHub," they mean the project's files and complete change history are stored on github.com.

You'll need a GitHub account. Go to github.com and sign up. The free tier is more than enough for everything we'll cover. You can have unlimited public and private repositories, and you can collaborate with as many people as you want.

One important distinction: Git is the tool. GitHub is a service that hosts Git repositories. They're related but separate. There are other hosting services like GitLab and Bitbucket, but GitHub is by far the most popular, and it's what most AI tools integrate with.

The Four Commands That Matter

Git has dozens of commands and hundreds of options. You need about four of them to be productive. Here's the workflow:

git init: Starting a New Project

When you start a new project, you tell Git to start tracking it:

```
git init
```

This creates a hidden `.git` folder in your project directory. That folder is where Git stores the entire history of your project. You run this command once, at the beginning.

If you're cloning a project from GitHub instead of starting fresh, you'd use `git clone` followed by the repository URL. That downloads the project and its full history to your machine.

git add: Choosing What to Save

Git doesn't automatically track every change you make. You explicitly tell it which changes you want to include in your next save point. This is called "staging."

```
git add index.js  
git add styles.css
```

Or, to stage everything that's changed:

```
git add .
```

Why does Git make you choose? Because sometimes you're working on two different things at once, and you want to save them separately. Maybe you fixed a bug and also started a new feature. You'd stage and save the bug fix first, then stage and save the feature work second. Clean history, clear intent.

git commit: Saving a Snapshot

Once you've staged your changes, you save them with a commit:

```
git commit -m "Fix login button not responding on mobile"
```

A commit is a snapshot of your project at that moment. The `-m` flag lets you attach a message explaining what changed and why. Good commit messages are a gift to your future self. "Fixed stuff" tells you nothing six months later. "Fix login button not responding on mobile devices due to touch event handler" tells you everything.

Each commit gets a unique identifier, a long string of letters and numbers (like `a1b2c3d`). You can use this to go back to any commit in your project's history.

git push: Sending to GitHub

Commits live on your machine until you push them:

```
git push
```

This sends your commits to GitHub. Now your code is backed up online, and anyone else working on the project can pull your changes down to their machine.

The full workflow, the one you'll repeat dozens of times a day, looks like this:

1. Make changes to your code
2. `git add .` to stage the changes
3. `git commit -m "Describe what you changed"` to save a snapshot
4. `git push` to send it to GitHub

That's it. Four commands. Everything else is refinement.

Branches: Safe Experimentation

Here's a scenario. Your app is working. Users are using it. You want to add a new feature, a search bar. But the search feature will take a few days to build, and during that time the code will be in a half-finished state. You don't want to break the working app.

Branches solve this. A branch is a parallel copy of your project where you can make changes without affecting the main version. Think of it like making a photocopy of a document, scribbling all over the copy, and only replacing the

original when you're happy with the scribbles.

```
git checkout -b add-search-bar
```

This creates a new branch called `add-search-bar` and switches you to it. Now every commit you make happens on this branch. The `main` branch (the original) stays exactly as it was.

You can have as many branches as you want. You can switch between them freely:

```
git checkout main          # go back to the main branch
git checkout add-search-bar # go back to your feature branch
```

When your feature is done and working, you merge it back into `main`. All the commits you made on the feature branch get incorporated into the main branch, and the project moves forward.

The standard workflow in professional development is: never work directly on `main`. Always create a branch for your work. This keeps the main branch stable and deployable at all times. If your feature branch turns out to be a dead end, you just delete it. No harm done.

Pull Requests: How Code Gets Reviewed

When you're working alone, you could merge branches directly. But in professional settings, and even in solo projects where you want to be careful, you use pull requests (often shortened to "PR").

A pull request is a formal way of saying: "I've made these changes on a branch, and I'd like to merge them into the main branch. Please review them first."

Here's how it works:

1. You push your branch to GitHub
2. On GitHub, you open a pull request

3. The PR shows exactly what changed: every file, every line added or removed
4. Reviewers can leave comments, ask questions, suggest modifications
5. Once approved, the branch gets merged into main

Pull requests are one of the most valuable practices in software development, even for solo builders. When you open a PR, you're forced to look at your own changes as a coherent unit. You often catch issues you missed while coding. The PR description becomes documentation of why you made the change.

If you're using Claude Code, you can ask it to open a pull request for you:

```
Create a pull request for the current branch with a description of what changed
```

Claude Code will write the PR title and description, push the branch, and create the PR on GitHub. You should still review what it wrote before merging.

How Claude Code Uses Git

Claude Code is deeply integrated with Git. Understanding what it's doing under the hood will save you from confusion and mistakes.

When you start a session and make changes, Claude Code will often commit for you. It creates commits with descriptive messages explaining what it changed. This is generally helpful, but you need to pay attention. Sometimes Claude Code makes multiple small commits when you'd prefer one. Sometimes the commit messages are overly verbose. These aren't problems exactly, but they're things to be aware of.

Here's what Claude Code does with Git:

It commits frequently. Claude Code tends to commit after completing a logical unit of work. Asked it to add a login page? It'll commit when the page is done. This is actually good practice, but it means your Git history might have more commits than you'd create manually.

It creates branches when asked. If you tell Claude Code to work on a feature, you can ask it to create a branch first. This is a good habit to build.

It can show you diffs. Before committing, you can ask Claude Code to show you what changed. This is the equivalent of reviewing your own work before saving.

It pushes when you tell it to. Claude Code won't automatically push to GitHub unless you ask it to. Your commits stay local until you explicitly say to push.

The key thing to understand: Claude Code is using the same Git commands you would use manually. It's not doing anything magical. It's running `git add`, `git commit`, and `git push` on your behalf. If something goes wrong, you can fix it with the same Git commands you'd use to fix any other Git problem.

One practical tip: at the start of a work session, tell Claude Code to create a new branch. Something like:

```
Create a new branch called feature/user-dashboard and switch to it
```

Now all the work in that session happens on a branch. If things go sideways, your main branch is untouched.

Common Git Mistakes and How to Fix Them

Git is forgiving. Almost everything can be undone. Here are the situations that trip people up most often, and how to get out of them.

"I committed something I shouldn't have"

Maybe you accidentally committed a file containing passwords or API keys. If you haven't pushed yet:

```
git reset HEAD~1
```

This undoes the last commit but keeps your changes. You can then remove the problematic file, add it to `.gitignore` (a file that tells Git to ignore certain files), and commit again.

If you've already pushed, the situation is more complex. The file is in GitHub's history. You'll need to rotate any exposed credentials immediately, because even if you remove the file in a new commit, the old commit still contains it. GitHub has tools to purge files from history, but the first priority is always changing the exposed credentials.

"I'm on the wrong branch"

You started making changes on `main` when you meant to be on a feature branch. If you haven't committed yet:

```
git stash
git checkout -b my-feature-branch
git stash pop
```

`git stash` temporarily shelves your changes. You create the branch you wanted, then `git stash pop` brings the changes back. No work lost.

"I have merge conflicts"

This happens when two branches changed the same lines in the same file. Git doesn't know which version to keep, so it asks you to decide.

The conflicted file will contain markers like this:

```
<<<<<<< HEAD
This is the text on your branch
=====
This is the text on the other branch
>>>>>>> other-branch
```

You edit the file to keep what you want, remove the markers, then stage and commit. Claude Code can help with merge conflicts. Tell it which file has conflicts and what the intended behavior should be, and it can resolve them for you.

"I want to undo everything and go back to the last working state"

If you've been experimenting and things are broken:

```
git stash
```

This removes all uncommitted changes but saves them in case you want them back. If you want to go back to a specific commit:

```
git log --oneline
```

This shows your commit history with short identifiers. Find the commit you want to return to and:

```
git checkout <commit-hash>
```

This puts you in a "detached HEAD" state, which sounds scary but just means you're looking at a historical snapshot. You can create a branch from here if you want to start fresh from that point.

"My .gitignore isn't working"

The .gitignore file tells Git which files to ignore: things like node_modules/ (thousands of dependency files), .env (environment variables with secrets), and .DS_Store (Mac system files). If you added a file to .gitignore but Git is still tracking it, that's because Git was already tracking it before you added the ignore rule. You need to untrack it:

```
git rm --cached .env  
git commit -m "Remove .env from tracking"
```

The file stays on your computer but Git stops tracking it going forward.

The .gitignore File

Every project needs a .gitignore file. This tells Git which files and folders to skip. For a typical Node.js project:

```
node_modules/  
.env  
.env.local  
.DS_Store  
dist/  
*.log
```

The `node_modules` folder can contain tens of thousands of files. You don't commit it because anyone can recreate it by running `npm install`. The `.env` file contains secrets like API keys and database passwords. Never commit secrets. The `dist` folder contains built output that gets regenerated. Log files are noise.

Claude Code will usually create a `.gitignore` for you when it sets up a project. Check it to make sure it includes `.env`, because that's the one that matters most for security.

A Practical Git Workflow

Here's what a real working day looks like with Git:

Morning: Pull the latest changes from GitHub (`git pull`). Create a branch for today's work (`git checkout -b feature/whatever`).

During the day: Make changes, commit regularly with clear messages. Each commit should represent a logical unit of work, not "saved at 3pm."

End of day: Push your branch to GitHub (`git push`). If the feature is done, open a pull request. If not, your work is safely backed up online.

When the feature is complete: Open a pull request. Review the changes. Merge into main. Delete the feature branch.

This workflow scales from a single person to a team of hundreds. The fundamentals don't change.

What You Now Know

You understand what Git does: it tracks every change to your project, letting you undo mistakes, experiment safely, and collaborate without chaos. You know the

four essential commands: `init`, `add`, `commit`, and `push`. You understand branches as a way to work on features without breaking what's already working, and pull requests as a way to review changes before they go live. You know how Claude Code uses Git on your behalf, and you know how to fix the most common Git mistakes. Most importantly, you understand that source control isn't optional. It's the foundation that makes everything else in professional software development possible.

Chapter 8: Authentication, Who Are You?

The moment your app has users, you have an authentication problem. Someone needs to sign up, sign in, prove they are who they say they are, and see only the things they're allowed to see. This sounds simple until you start thinking about password storage, session management, account recovery, brute force attacks, and a dozen other security concerns. The good news: you should never build this yourself. Services like Clerk handle all of it, and integrating them into your app is something Claude Code can do in minutes. This chapter explains what authentication actually involves, why it's harder than it looks, and how to add it to your Next.js app without losing sleep over security.

Why Authentication Is Hard

You might think authentication is just checking a username and password against a database. Type in your email, type in your password, if they match, you're in. How hard could it be?

Very hard, as it turns out. Here's a partial list of things you'd need to handle if you built authentication from scratch:

Password storage. You can never store passwords as plain text. If your database gets breached (and breaches happen to companies of every size), every user's password is exposed. Instead, you store a "hash," a mathematically scrambled version of the password that can't be reversed. But not all hashing algorithms are equal. MD5 and SHA-1 are broken. You need bcrypt, scrypt, or Argon2, and you need to use them correctly with proper salt values.

Session management. When a user logs in, you need to remember that they're logged in as they navigate from page to page. This is typically done with session

tokens or JWTs (JSON Web Tokens). You need to generate them securely, store them properly (httpOnly cookies, not localStorage), set appropriate expiration times, and invalidate them when the user logs out.

Account recovery. Users forget passwords. You need a "forgot password" flow: send a reset link to their email, make sure the link expires after a short time, make sure it can only be used once, make sure it's sent to the right email address.

Brute force protection. Someone could try thousands of password combinations against your login form. You need rate limiting, account lockouts after failed attempts, and possibly CAPTCHA challenges.

Multi-factor authentication. Passwords alone aren't secure enough for many applications. You might need SMS codes, authenticator apps, or passkeys.

OAuth and social login. Users expect to sign in with Google, Apple, or GitHub. Each provider has its own API, its own token format, and its own quirks.

Email verification. You need to verify that users actually own the email address they signed up with. This means sending verification emails, handling confirmation links, and deciding what users can access before they verify.

This is just the security side. You also need the UI: sign-up forms, sign-in forms, password reset pages, account settings, user profile management. Each of these needs proper validation, error messages, and accessibility.

Building all of this correctly takes an experienced security engineer weeks or months. Getting any of it wrong creates vulnerabilities that could expose your users' data. This is why the professional approach is to use a dedicated authentication service.

Why You Should Never Build Your Own Auth

This advice isn't just for non-developers. Even experienced engineering teams at well-funded companies use third-party auth services. The reasoning is straightforward: authentication is a solved problem with severe consequences for getting it wrong.

When you build your own auth:

- You become responsible for security updates as new vulnerabilities are discovered
- You need to stay current with evolving best practices (password hashing standards change over time)
- You're liable for any data breach caused by an auth vulnerability
- You spend development time on plumbing instead of features that differentiate your product

When you use an auth service:

- Security is maintained by a dedicated team of specialists
- Compliance requirements (SOC 2, GDPR) are handled for you
- Features like multi-factor auth, social login, and passwordless options come built-in
- You can focus on what makes your app unique

The one exception is if you're building a product where authentication is the core business, like a password manager or identity platform. For everything else, use a service.

Clerk: The Simplest Auth for Modern Apps

There are several auth services available: Auth0, Firebase Auth, Supabase Auth, Clerk, and others. We're using Clerk in this book because it has the lowest friction for Next.js projects and provides the best developer experience for someone working with AI coding tools.

Clerk gives you:

- **Pre-built UI components:** Sign-up forms, sign-in forms, and user profile pages that look good out of the box
- **React components and hooks:** Drop `<SignIn />` into your page and you have a complete sign-in flow

- **Middleware for route protection:** Define which pages require authentication with a few lines of configuration
- **User management dashboard:** See all your users, manage their accounts, and configure settings through a web interface
- **Social login:** Google, GitHub, Apple, and others with minimal configuration
- **Webhooks:** Get notified when users sign up, sign in, or update their profiles

The free tier supports up to 10,000 monthly active users, which is more than enough for most internal tools and early-stage products.

Setting Up Clerk

Here's the process, step by step.

1. Create a Clerk Account

Go to clerk.com and sign up. Create an application. Clerk will ask you what sign-in methods you want to support. For most apps, start with email/password and Google. You can add more later.

2. Get Your API Keys

Clerk gives you two keys:

- `CLERK_PUBLISHABLE_KEY`, used in the browser, safe to expose in client-side code
- `CLERK_SECRET_KEY`, used on the server, must never be exposed

Add both to your `.env.local` file:

```
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_your_key_here
CLERK_SECRET_KEY=sk_test_your_key_here
```

Remember from Chapter 7: your `.env.local` file should be in your `.gitignore`. Never commit API keys to Git.

3. Install the Clerk Package

```
npm install @clerk/nextjs
```

4. Wrap Your App with ClerkProvider

In your root layout file (`app/layout.tsx` or `app/layout.js`), wrap everything with the `ClerkProvider` component:

```
import { ClerkProvider } from '@clerk/nextjs'

export default function RootLayout({ children }) {
  return (
    <ClerkProvider>
      <html lang="en">
        <body>{children}</body>
      </html>
    </ClerkProvider>
  )
}
```

This makes Clerk's authentication context available throughout your app. Every page can now check whether the user is signed in and who they are.

5. Add Middleware for Route Protection

Create a `middleware.ts` file in your project root:

```
import { clerkMiddleware } from '@clerk/nextjs/server'

export default clerkMiddleware()

export const config = {
  matcher: [
    '/((?!_next|[^?]*\\.(?:html?|css|js(?!on)|jpe?g|webp|png|gif|svg|ttf|woff2?|ic)',
    '/(api|trpc)(.*)',
  ],
}
```

This middleware runs on every request and checks authentication status. By default, all routes are public. You make routes private by explicitly protecting them.

6. Create Sign-In and Sign-Up Pages

Clerk provides pre-built components:

```
// app/sign-in/[...sign-in]/page.jsx
import { SignIn } from '@clerk/nextjs'

export default function SignInPage() {
  return (
    <div style={{ display: 'flex', justifyContent: 'center', padding: '2rem' }}>
      <SignIn />
    </div>
  )
}
```

```
// app/sign-up/[...sign-up]/page.jsx
import { SignUp } from '@clerk/nextjs'

export default function SignUpPage() {
  return (
    <div style={{ display: 'flex', justifyContent: 'center', padding: '2rem' }}>
      <SignUp />
    </div>
  )
}
```

Add the routes to your `.env.local`:

```
NEXT_PUBLIC_CLERK_SIGN_IN_URL=/sign-in
NEXT_PUBLIC_CLERK_SIGN_UP_URL=/sign-up
```

That's the basic setup. You now have working authentication: users can create accounts, sign in, sign out, and reset their passwords. Clerk handles all the security concerns we discussed earlier.

Protecting Pages

Now that auth is set up, you need to decide which pages require a logged-in user. There are two approaches.

Protecting Individual Pages

In any server component, you can check authentication:

```
import { auth } from '@clerk/nextjs/server'
import { redirect } from 'next/navigation'

export default async function DashboardPage() {
  const { userId } = await auth()

  if (!userId) {
    redirect('/sign-in')
  }

  return <div>Welcome to your dashboard</div>
}
```

If the user isn't signed in, they get redirected to the sign-in page. If they are, they see the dashboard.

Protecting Groups of Routes

For a more scalable approach, you can configure the middleware to protect entire sections of your app. Update your `middleware.ts`:

```
import { clerkMiddleware, createRouteMatcher } from '@clerk/nextjs/server'

const isProtectedRoute = createRouteMatcher([
  '/dashboard(.*)',
  '/settings(.*)',
  '/api/private(.*)',
])

export default clerkMiddleware(async (auth, req) => {
  if (isProtectedRoute(req)) {
    await auth.protect()
  }
})
```

Now every URL starting with `/dashboard`, `/settings`, or `/api/private` requires authentication. Everything else is public. This is cleaner than checking auth in every individual page.

User Management: Roles and Permissions

Most apps need more than just "signed in" or "not signed in." You might have admins who can manage other users, editors who can modify content, and viewers who can only read. This is authorization, which is different from authentication. Authentication is proving who you are. Authorization is determining what you're allowed to do.

Clerk supports this through organizations and roles. For simpler cases, you can use Clerk's user metadata to assign roles:

```
// Checking if a user is an admin
import { auth } from '@clerk/nextjs/server'

export default async function AdminPage() {
  const { userId, sessionClaims } = await auth()

  if (sessionClaims?.metadata?.role !== 'admin') {
    return <div>Access denied</div>
  }

  return <div>Admin panel content</div>
}
```

You set user roles through Clerk's dashboard or API. In the Clerk dashboard, you can click on any user, go to their metadata, and add a role field. For production apps, you'd typically set roles programmatically when users are created or when an admin promotes them.

For more complex permission systems, Clerk offers Organizations, which let you create teams with defined roles (admin, member, guest) and granular permissions. This is useful for B2B apps where different companies have their own user groups.

Adding Clerk with Claude Code

Here's where AI-assisted development shines. Instead of copying code snippets from documentation and figuring out where they go, you can tell Claude Code what you want:

Add Clerk authentication to this Next.js app. I need:

- Sign in and sign up pages
- A protected dashboard route
- A user button in the navigation bar
- Middleware that protects all /dashboard routes

Claude Code will install the package, create the necessary files, update your layout, create the middleware, and wire everything together. It knows Clerk's current API because it can reference the documentation.

A few things to keep in mind when working with Claude Code on auth:

Always provide your API keys through `.env.local`, not in the prompt. Don't paste your secret key into a conversation with Claude Code. Create the `.env.local` file manually or tell Claude Code the key names and add the values yourself.

Test the auth flow manually. After Claude Code sets up Clerk, go through the full flow yourself: sign up for a new account, verify your email, sign in, visit a protected page, sign out, try to access the protected page while signed out. Each step should work correctly.

Check the middleware carefully. The middleware configuration determines which routes are public and which are private. A mistake here means either everything is locked down (users can't access your public pages) or nothing is protected (anyone can access private data).

The Auth Checklist: Before Going Live

Before you deploy an app with authentication to real users, verify each of these:

Environment variables are set in production. Your `.env.local` file is only for local development. When you deploy to Railway or another hosting service, you need to set the same environment variables there. In Railway, go to your service settings and add the Clerk keys.

The sign-up flow works end to end. Create a test account. Verify the email. Sign in. Make sure the user appears in your Clerk dashboard.

Protected routes actually protect. Open a private browser window (where you're not signed in) and try to access a protected URL directly. You should be

redirected to the sign-in page.

The sign-out flow works. Sign out and confirm you're redirected appropriately. Make sure you can't access protected pages after signing out by pressing the back button.

Error states are handled. What happens if someone enters a wrong password? What if the email is already taken during sign-up? What if the Clerk service is temporarily unavailable? Test these scenarios.

HTTPS is enabled. Authentication should only happen over HTTPS. Railway provides HTTPS by default. If you're using a custom domain, make sure SSL is configured.

Session timeout is appropriate. Clerk's default session duration works for most apps, but if you're building something sensitive (financial data, health records), you might want shorter sessions. This is configurable in the Clerk dashboard.

You're using Clerk's production keys. Clerk gives you separate keys for development and production. Development keys only work on localhost. Before deploying, switch to your production keys in your hosting environment.

Beyond Basic Auth

Once basic authentication is working, there are features you might want to add over time:

Social login options. In the Clerk dashboard, enable additional providers like GitHub, Apple, or Microsoft. Users can then sign in with accounts they already have.

Multi-factor authentication. Clerk supports TOTP (authenticator apps) and SMS verification. You can require MFA for all users or let them opt in.

Custom sign-in pages. Clerk's pre-built components are functional but generic. You can customize their appearance through Clerk's theming system or build completely custom UI using Clerk's hooks and API.

Webhooks for user events. Want to send a welcome email when someone signs up? Want to update your database when a user changes their profile?

Clerk can send webhooks to your API endpoints when these events occur.

None of these require you to understand the underlying security mechanisms. That's the entire point of using a service. You configure what you want through a dashboard or a few lines of code, and the security team at Clerk handles the implementation.

Common Auth Mistakes to Avoid

Even when using a service like Clerk, there are mistakes that can compromise your app's security or create a frustrating user experience.

Forgetting to protect API routes. You protect your pages so users can't access the dashboard without signing in. But your API endpoints, the URLs that return data, are unprotected. Someone discovers `/api/users` and can pull your entire user list without authentication. Every API route that returns or modifies sensitive data needs an auth check.

Exposing user IDs in URLs. If your app uses URLs like `/users/123/profile`, someone might try changing 123 to 124 to see another user's profile. This is called an IDOR (Insecure Direct Object Reference) vulnerability. Always verify that the authenticated user has permission to view the requested resource, not just that they're signed in.

Storing tokens in the wrong place. If you're handling any auth tokens manually (you usually won't with Clerk, but it comes up), never store them in `localStorage`. It's accessible to any JavaScript running on your page, including malicious scripts injected through cross-site scripting (XSS) attacks. Clerk handles token storage correctly with `httpOnly` cookies, which is one more reason to use it instead of rolling your own solution.

Not handling the signed-out state everywhere. You test the happy path: user signs in, sees their dashboard, everything works. But what about the loading state while Clerk checks authentication? What about components deep in your app that assume a user is always present? If `user` is null and your code tries to access `user.email`, your app crashes. Always account for the possibility that the user isn't signed in, even on pages that should be protected.

Ignoring the development-to-production key switch. Clerk's development keys are convenient: they work on localhost, they don't send real emails, and

they're forgiving with domain restrictions. But they don't work in production. A common launch-day issue is deploying with development keys and wondering why nobody can sign in.

What You Now Know

You understand why authentication is genuinely difficult, covering password hashing, session management, account recovery, and brute force protection. You know why building your own auth system is almost always the wrong choice, even for experienced developers. You've seen how Clerk provides a complete authentication solution for Next.js apps, from pre-built sign-in components to middleware-based route protection. You know how to set up Clerk in a project, protect routes, manage user roles, and use Claude Code to accelerate the integration. And you have a concrete checklist to verify before deploying authentication to real users. The key takeaway: authentication is infrastructure, not a feature you build. Use a service, focus on your app, and sleep well knowing your users' credentials are handled by specialists.

Chapter 9: Reviews, Testing, and Quality

Getting your app to work is the first milestone, not the finish line. Software that works in the demo but crashes when a real user clicks the wrong button, enters unexpected data, or uses it on a slow connection isn't ready for production. This chapter covers the practices that separate "it works on my machine" from "it works for everyone, every time." You don't need to become a QA engineer. But you need to understand code review, testing, and error handling well enough to catch the problems that AI-generated code won't catch by itself.

Why "It Works" Isn't Enough

When Claude Code builds a feature for you and you see it working in your browser, it's tempting to call it done. The button clicks, the data saves, the page loads. Ship it.

But "it works" usually means "it works in the one scenario I tested." Software fails in the margins. The situations nobody thought about. Here are real examples of how things break:

The empty state. Your dashboard looks great with 10 items. What does it look like with zero items? Does it show a helpful message or a blank white screen? Does it crash because the code assumes there's always at least one item?

The overload. Your list works fine with 10 entries. What about 10,000? Does the page take 30 seconds to load? Does the browser freeze? Do the pagination controls work?

The bad input. Your form accepts a name. What if someone enters a name that's 500 characters long? What if they paste HTML or JavaScript into the

field? What if they submit the form with nothing filled in?

The slow network. You're testing on your fast office Wi-Fi. Your users might be on a phone with spotty reception. Does the app show a loading indicator, or does it appear broken while waiting for data?

The concurrent user. Two people edit the same record at the same time. Whose changes win? Does the app handle this gracefully or silently discard someone's work?

The expired session. A user leaves the tab open overnight. Their authentication session expires. When they come back and click a button, does the app redirect them to sign in, or does it show a cryptic error?

These aren't hypothetical. Every production app encounters these scenarios. The question is whether you've handled them before or after users start complaining.

Code Review: What to Look For

Code review is the practice of reading through code changes before they go live. In professional development teams, no code ships without at least one other person reviewing it. This catches bugs, improves code quality, and spreads knowledge across the team.

As a non-developer using AI tools, you might think code review isn't for you. But you can, and should, review what Claude Code produces. You don't need to understand every line. You need to look for patterns that signal problems.

Structure and Organization

Open the files that changed. Even without understanding the code, you can spot red flags:

- **Files that are extremely long.** If a single file is 500+ lines, it's probably doing too much. Well-organized code splits responsibilities across multiple files.
- **Repeated code.** If you see the same block of code copy-pasted in multiple places, that's a maintenance problem. When you need to change that logic, you'll need to change it everywhere.

- **Inconsistent naming.** If some functions are called `getUserData` and others are called `fetch_user_info`, the codebase lacks consistency. This makes it harder to understand and maintain.

Security Concerns

You can check for security issues without being a security expert:

- **Are API keys or passwords in the code?** Search the changed files for anything that looks like a key, token, or password. These should always be in environment variables.
- **Is user input being trusted?** If user-entered data is inserted directly into database queries or HTML without sanitization, that's a vulnerability.
- **Are auth checks in place?** For any page or API endpoint that should be private, verify that there's an authentication check.

Logic and Completeness

Ask yourself these questions about every feature:

- What happens when there's no data?
- What happens when there's too much data?
- What happens when the user enters something unexpected?
- What happens when the network request fails?
- Does the UI show loading states while waiting for data?
- Are error messages helpful to the user?

You don't need to know how to fix these issues. You need to know how to spot them and tell Claude Code to fix them.

Using Claude Code for Automated Review

Claude Code has a built-in review capability. After making changes, you can ask for a review:


```
Review the changes I just made. Check for bugs, security issues, and edge cases I
```

Claude Code will look through the modified files and point out potential issues. This is genuinely useful, but it's not a replacement for your own review. AI tends to focus on code-level issues (unused variables, potential null references) and may miss higher-level concerns (does this feature actually solve the user's problem? Is the workflow intuitive?).

For a more thorough review, you can use the `/review` command, which applies a structured review process. It checks for:

- Security vulnerabilities
- Performance concerns
- Error handling gaps
- Code organization issues
- Potential bugs

Think of automated review as a second pair of eyes, not the only pair. Use it after your own review to catch things you missed.

Getting the Most from AI Reviews

To get a useful review from Claude Code, be specific about what you're concerned about:

```
Review the authentication flow in the dashboard. I'm worried about:  
1. What happens if the session expires mid-action  
2. Whether all API routes are properly protected  
3. Whether error messages might leak sensitive information
```

A targeted review question gets a targeted, useful answer. A generic "review my code" gets a generic response.

Testing Basics

Testing in software development means writing code that verifies your other code works correctly. That sounds circular, but it's practical. Instead of manually clicking through your app every time you make a change, you write automated tests that do the clicking for you.

There are three levels of testing that matter:

Unit Tests

A unit test verifies that a single function does what it's supposed to do. If you have a function that calculates a discount:

```
function calculateDiscount(price, discountPercent) {  
  return price - (price * discountPercent / 100)  
}
```

A unit test would verify:

```
test('calculates 20% discount on $100', () => {  
  expect(calculateDiscount(100, 20)).toBe(80)  
})  
  
test('handles zero discount', () => {  
  expect(calculateDiscount(100, 0)).toBe(100)  
})  
  
test('handles 100% discount', () => {  
  expect(calculateDiscount(100, 100)).toBe(0)  
})
```

Each test checks one scenario. If someone later changes the `calculateDiscount` function and breaks it, the test fails immediately, before the broken code reaches users.

You can ask Claude Code to write unit tests:

```
Write unit tests for the calculateDiscount function in utils/pricing.js.  
Cover normal cases, edge cases, and invalid inputs.
```

Integration Tests

Integration tests verify that different parts of your app work together. A unit test checks that the discount function calculates correctly. An integration test checks that when a user adds an item to their cart and applies a coupon code, the correct discounted price shows up on the checkout page.

These tests are more complex to write and maintain, but they catch a different class of bugs, the ones where individual pieces work fine in isolation but fail when combined.

End-to-End Tests

End-to-end (E2E) tests simulate a real user interacting with your app in a browser. They click buttons, fill in forms, navigate between pages, and verify that everything works as expected. Tools like Playwright and Cypress automate this.

An E2E test might look like:

```
test('user can sign in and view dashboard', async ({ page }) => {
  await page.goto('/sign-in')
  await page.fill('[name="email"]', 'test@example.com')
  await page.fill('[name="password"]', 'testpassword')
  await page.click('button[type="submit"]')
  await expect(page).toHaveURL('/dashboard')
  await expect(page.locator('h1')).toContainText('Welcome')
})
```

For a non-developer building with AI tools, E2E tests are arguably the most valuable type. They test what the user actually experiences. You can describe the test scenario in plain English and have Claude Code write the test:

```
Write an E2E test that:
1. Goes to the sign-in page
2. Signs in with test credentials
3. Navigates to the settings page
4. Changes the user's display name
5. Verifies the new name appears on the dashboard
```

How Much Testing Is Enough?

Professional software teams aim for high test coverage, meaning most of the code is verified by at least one test. For an AI-assisted builder shipping internal tools or early-stage products, you don't need 100% coverage. Focus on:

- **Critical paths.** Sign-in, sign-up, the core action your app performs (creating an order, submitting a report, generating a document). If these break, your app is useless.
- **Data handling.** Anywhere data is created, modified, or deleted. These are the most consequential operations.
- **Edge cases you've already encountered.** Every time you find a bug, write a test that would catch it. This prevents the same bug from coming back.

Start with a handful of E2E tests covering your app's main workflow. Add more as you discover issues. Some tests are infinitely better than no tests.

Error Handling: Planning for Failure

Every app talks to external services: databases, APIs, authentication providers, payment processors. These services can be slow, unavailable, or return unexpected data. Your app needs to handle these situations gracefully.

The User's Perspective

When something goes wrong, users need three things:

1. **To know something went wrong.** A silent failure where nothing happens is worse than an error message. The user doesn't know if they should wait, retry, or give up.
2. **To understand what went wrong, in simple terms.** "An error occurred" is barely better than silence. "We couldn't save your changes because the server is temporarily unavailable" tells the user what happened.
3. **To know what to do next.** "Please try again in a few minutes" or "Contact support at help@example.com" gives the user a path forward.

Common Error Scenarios

Here are the failures your app should handle:

Network errors. The API call fails because the user's internet dropped, or the server is down. Show a clear message and offer a retry button.

```
try {
  const response = await fetch('/api/data')
  if (!response.ok) {
    throw new Error('Server error')
  }
  const data = await response.json()
  // use the data
} catch (error) {
  // show user-friendly error message
  showError('Unable to load your data. Please check your connection and try again.')
}
```

Validation errors. The user submits a form with missing or invalid fields. Highlight the specific fields that need attention and explain what's expected. Don't just say "Invalid input." Say "Email address is required" or "Password must be at least 8 characters."

Permission errors. The user tries to access something they're not authorized for. Show a clear "access denied" message, not a blank page or a crash.

Not found errors. The user navigates to a URL that doesn't exist, maybe a bookmarked page for an item that's been deleted. Show a helpful 404 page that guides them back to a working part of the app.

Rate limiting. If your app makes too many API calls too quickly, the API might reject requests. Handle this by queuing requests or showing the user a "please wait" message.

Asking Claude Code to Add Error Handling

After building a feature, explicitly ask Claude Code to add error handling:

```
Add thorough error handling to the data submission flow:
- What if the API is down?
- What if the user's session has expired?
- What if the submitted data fails validation?
- Show user-friendly error messages for each case
- Add a retry mechanism for network failures
```

Claude Code is good at this. It knows the common patterns for error handling in JavaScript and React. But it won't always add error handling unless you ask for it. When building features, it tends to focus on the "happy path," the scenario where everything works perfectly. Your job is to remind it that things break.

Quality as a Habit

The biggest mistake in software quality is treating it as a phase. Build first, test later, fix bugs before launch. This never works. By the time you're "testing," you have a pile of interconnected code where fixing one thing breaks another.

Quality works better as a continuous practice:

Review as you go. After Claude Code implements a feature, review it immediately. Check the edge cases. Test the error states. Don't wait until you have ten features to review.

Test the critical path after every change. Made a change to the database schema? Run through the sign-in flow, the main feature, and the data entry process. Make sure nothing broke.

Fix bugs before adding features. When you find a bug, fix it before moving on. Bugs compound. A small bug in the data layer causes a bigger bug in the business logic, which causes a confusing error in the UI.

Keep a testing checklist. Write down the scenarios you test before each deployment. This takes five minutes and prevents the "I forgot to test the sign-up flow" disasters.

Here's a simple checklist template:

```
Pre-deployment checklist:
[ ] Sign up with a new account
[ ] Sign in with existing account
[ ] Access the main feature (create/read/update/delete)
[ ] Test with empty data
[ ] Test with invalid input
[ ] Verify protected routes redirect to sign-in
[ ] Check on mobile screen size
[ ] Check with slow network (browser dev tools can simulate this)
[ ] Review error messages for user-friendliness
```

You don't need sophisticated testing infrastructure. You need the discipline to go through these checks every time, and to add new checks when you discover new ways things can break.

The Cost of Skipping Quality

It's worth understanding what happens when quality practices are skipped, because the consequences aren't always obvious.

Compound bugs. You ship a feature with a small data formatting issue. It's minor, dates display as "2026-4-16" instead of "April 16, 2026." You plan to fix it later. Next week, you build a reporting feature that parses those dates. It expects the formatted version and breaks on the raw format. Now you have two bugs instead of one, and fixing the first requires updating the second.

Lost user trust. Users forgive the occasional error if your app generally works well. But if they encounter multiple issues in their first session, they form a judgment: this app is unreliable. Changing that perception is much harder than preventing it. First impressions in software are expensive.

Debugging difficulty. When you ship multiple features without testing between them, and something breaks, you don't know which change caused the problem. You're now debugging a much larger surface area. If you'd tested after each change, you'd know exactly where the issue was introduced.

Scope creep through assumptions. When you don't review what Claude Code builds, you might not notice that it added features you didn't ask for. Maybe it built an elaborate admin panel when you asked for a simple settings page. Maybe it added database columns you don't need. These extras increase complexity and maintenance burden without adding value.

The time you invest in quality practices, reviewing code, writing tests, handling errors, always pays for itself. Not eventually. Immediately. Every bug you catch before deployment is a bug you don't have to debug in production, explain to users, or fix under pressure.

What You Now Know

You understand why "it works" is the starting point, not the finish line, and you know the categories of failures that catch people off guard: empty states, bad input, slow networks, expired sessions. You can review code for structural issues, security concerns, and missing edge cases even without deep programming knowledge. You know how to use Claude Code's review capabilities effectively by asking targeted questions. You understand the three levels of testing, unit, integration, and end-to-end, and know that starting with a few E2E tests on your critical path is more valuable than theoretically perfect coverage. You know how to handle errors in a way that respects your users by telling them what went wrong and what to do about it. Most importantly, you understand that quality isn't something you bolt on at the end. It's a habit you practice with every change, every feature, every deployment.

Chapter 10: Spec-Driven Development

If you've followed this book from the beginning, you've gone from opening an HTML file in a browser to deploying a full-stack app with authentication, a database, and source control. That's a real achievement. But there's a gap between building something that works and building something that lasts. This final chapter is about crossing that gap. Spec-driven development is the practice of defining what you're building before you build it, giving your AI tools clear instructions instead of improvising, and treating your project like a product rather than an experiment. It's where vibe coding grows up.

The Problem with Vibe Coding

Vibe coding is how most people start with AI development tools. You open Claude Code or Cursor, describe what you want in natural language, and the AI builds it. It's fast, it's exciting, and it produces results that feel like magic. You say "build me a dashboard with user analytics" and minutes later you have something on screen.

The problem shows up around week three.

You want to change how the analytics are calculated, but the AI built the logic in four different places and you're not sure which one is the source of truth. You want to add a new feature, but the new code conflicts with assumptions the AI made earlier. You describe what you want, and the AI produces something that looks right but behaves differently from what you expected because your description was ambiguous.

Vibe coding breaks down because natural language is imprecise. When you say "build me a user dashboard," you have a picture in your head of what that

means. The AI has a different picture. The gap between those pictures creates bugs, inconsistencies, and rework.

This isn't the AI's fault. It's doing its best with the information you gave it. The fix isn't to stop using AI. It's to give it better information.

Writing a Spec Before Writing Code

A specification (spec) is a document that describes what you're building in enough detail that there's little room for misinterpretation. It's not a novel. It's a clear, structured description of the what, why, how, and constraints of a feature or project.

Here's what a good spec covers:

What Are We Building?

A concrete description of the feature or system. Not "a dashboard" but "a page that shows the admin a list of all active users, their sign-up date, last login date, and current subscription tier. The list should be sortable by any column and filterable by subscription tier."

See the difference? The second description leaves much less room for interpretation. The AI knows exactly what columns to show, what interactions to support, and who the audience is.

Why Are We Building It?

The business reason. "The admin currently has to query the database directly to see user information. This page eliminates that dependency and lets non-technical team members check user status." This context helps the AI make better decisions when it encounters ambiguity. If it knows the audience is non-technical, it'll prioritize clarity over power-user features.

How Should It Work?

The expected behavior, step by step. "When the page loads, it fetches all users from the `/api/users` endpoint and displays them in a table. The default sort is by sign-up date, newest first. Clicking a column header sorts by that column. Clicking again reverses the sort. The tier filter is a dropdown above the table

with options: All, Free, Pro, Enterprise."

What Are the Constraints?

Limitations and requirements that aren't obvious. "The page must load within 2 seconds even with 10,000 users. Pagination should show 50 users per page. The page should be accessible only to users with the admin role. The table should be responsive on mobile, showing at minimum the user name and tier."

What Are We Not Building?

Sometimes the most useful part of a spec is what it explicitly excludes. "This version does not include user editing, bulk actions, or export functionality. Those are planned for a future iteration." This prevents the AI from over-building and keeps the scope manageable.

CLAUDE.md as Your Project's Operating Manual

You've seen CLAUDE.md mentioned earlier in this book. It's a file in your project's root directory that Claude Code reads automatically at the start of every session. Think of it as the operating manual for your project, a document that tells the AI how your project works, what conventions to follow, and what to avoid.

A good CLAUDE.md contains:

Project overview. What the app does, who it's for, and what technology stack it uses. This gives Claude Code context it would otherwise have to guess or ask about.

```
# Project: InventoryTracker
```

```
Internal tool for warehouse team to track product inventory.  
Built with Next.js 14, PostgreSQL, Clerk auth, deployed on Railway.
```

```
## Users
```

- Warehouse staff: scan items, update quantities
- Managers: view reports, approve orders
- Admins: manage users, configure settings

Technical conventions. How files are organized, how things are named, what patterns to use.

```
## Conventions
```

- All database queries go through /lib/db.ts
- API routes use the /api/v1/ prefix
- All pages require authentication except /sign-in and /sign-up
- Use server components by default, client components only when needed
- Error responses follow format: { error: string, code: number }

Do and don't rules. Specific instructions for what the AI should and shouldn't do.

```
## Rules
```

- Never modify the database schema without updating /docs/schema.md
- Always add error handling for API calls
- Never store sensitive data in localStorage
- Always use parameterized queries, never string concatenation for SQL
- Keep components under 200 lines, split if larger

Commands and workflows. How to run, test, and deploy the project.

```
## Development
```

- Run locally: npm run dev
- Run tests: npm test
- Deploy: push to main branch (auto-deploys via Railway)
- Database migrations: npm run db:migrate

The power of CLAUDE.md is consistency. Without it, every conversation with Claude Code starts from zero. You re-explain your stack, your conventions, your preferences. With it, Claude Code starts every session already understanding

your project. It follows your naming conventions, respects your file organization, and avoids the mistakes you've told it about.

AGENTS.md: Defining AI Boundaries

AGENTS.md is a newer convention that goes beyond CLAUDE.md. While CLAUDE.md describes the project, AGENTS.md describes how AI agents should behave when working on it.

This file is especially important as AI tools become more autonomous. When you give Claude Code a complex task, it might make dozens of decisions along the way. AGENTS.md helps ensure those decisions align with your intentions.

```
# AGENTS.md

## Scope
- Can create new files in /src and /components
- Can modify existing files
- Cannot delete files without explicit permission
- Cannot modify environment variables or deployment config

## Decision Making
- When unsure about a design choice, ask rather than guess
- Prefer simple solutions over clever ones
- If a task requires modifying more than 5 files, stop and outline the plan first

## Code Quality
- All new functions must include JSDoc comments
- All API endpoints must have error handling
- All database operations must use transactions where appropriate
- Run linting before committing

## Testing
- Write a test for every new API endpoint
- Update existing tests when modifying API behavior
- Never skip failing tests, fix them
```

Think of AGENTS.md as the rules of engagement. It's especially valuable when you're using multiple AI tools or when different team members interact with the AI. Everyone follows the same playbook.

DESIGN.md: Visual Consistency

If your app has a user interface, visual consistency matters. Without explicit design guidelines, each AI-generated page might look slightly different: different spacing, different button styles, different color choices.

DESIGN.md solves this by documenting your app's visual language:

```
# DESIGN.md

## Colors
- Primary: #2563EB (blue)
- Secondary: #64748B (slate)
- Success: #16A34A (green)
- Error: #DC2626 (red)
- Background: #FFFFFF
- Text: #1E293B

## Typography
- Headings: Inter, semi-bold
- Body: Inter, regular, 16px
- Code: JetBrains Mono, 14px

## Spacing
- Section padding: 2rem
- Card padding: 1.5rem
- Button padding: 0.5rem 1rem
- Gap between form fields: 1rem

## Components
- Buttons: rounded corners (8px), primary color for main actions, outlined for secondary
- Cards: white background, subtle border (#E2E8F0), 8px border radius, light shadow
- Tables: striped rows, sticky header, hover highlight
- Forms: labels above inputs, validation messages in red below the field

## Responsive Breakpoints
- Mobile: below 640px
- Tablet: 640px - 1024px
- Desktop: above 1024px
```

When Claude Code has this reference, every page it builds uses the same colors, spacing, and component styles. The app looks intentional and cohesive instead of assembled from different kits.

The Spec-Driven Workflow

Here's how everything comes together into a practical workflow:

Step 1: Write the Spec

Before asking Claude Code to build anything, write a spec for what you want. It doesn't need to be long. A few paragraphs covering what, why, how, and constraints is enough for most features. Save it as a markdown file in your project: `/specs/user-dashboard.md` or similar.

Step 2: AI Implements

Give Claude Code the spec. Tell it to read the spec file and implement it:

```
Read /specs/user-dashboard.md and implement the feature described there.  
Follow the conventions in CLAUDE.md and the design guidelines in DESIGN.md.
```

Because the spec is detailed and the conventions are documented, the AI produces code that's much closer to what you want on the first try. Less back-and-forth, fewer misunderstandings, less rework.

Step 3: Human Reviews

Review what Claude Code built. Check it against the spec. Does it do everything the spec describes? Does it follow the design guidelines? Are there edge cases it missed? Use the review practices from Chapter 9.

Step 4: Iterate

No implementation is perfect on the first pass. But with a spec as your reference point, feedback becomes specific and productive. Instead of "this doesn't look right," you can say "the spec says the table should be sortable by any column, but clicking the tier column doesn't sort." Specific feedback gets specific fixes.

Step 5: Update the Spec

As you learn from building, update the spec. Maybe you discover a requirement you didn't think of. Maybe a constraint changes. The spec is a living document,

not a contract carved in stone. Update it, and the next time Claude Code works on this feature, it has the latest information.

From Hobby Project to Production Software

There's a maturity curve in software projects, and understanding where you are on it helps you make appropriate decisions about how much process to apply.

Stage 1: Exploration

You're trying an idea. Vibe coding is perfectly appropriate here. Move fast, experiment, see if the concept has legs. Don't write specs. Don't worry about test coverage. You're exploring.

Stage 2: Validation

The idea works. People want it. You're building a version that real users will rely on. This is where you start writing specs for new features, create a CLAUDE.md, and add basic testing. You're not building for scale yet, but you're building to last.

Stage 3: Growth

Users depend on your app. You're adding team members or giving access to more people. This is where AGENTS.md, DESIGN.md, thorough testing, code review, and formal deployment processes become necessary. The cost of a bug goes up because more people are affected.

Stage 4: Production

The app is critical infrastructure. Changes go through pull requests, automated tests run on every commit, deployments are staged (test environment first, then production), and someone is monitoring for errors in real time. This is where most professional software lives.

You don't need to start at Stage 4. That would be over-engineering for an early idea. But you should recognize when your project has moved from one stage to the next and adjust your practices accordingly. The most common mistake is

staying at Stage 1 when your project is really at Stage 3: users depend on something you're still treating like an experiment.

The Maturity Checklist

Here's a practical checklist for evaluating where your project stands and what to add next:

Stage 1 (Exploration):

- ☐ Code in a Git repository
- ☐ .gitignore file in place
- ☐ Basic understanding of what you're building

Stage 2 (Validation):

- ☐ CLAUDE.md with project overview and conventions
- ☐ .env files for secrets, never committed
- ☐ Deployed to a hosting service (Railway)
- ☐ Basic error handling on critical paths
- ☐ Authentication if users log in

Stage 3 (Growth):

- ☐ AGENTS.md defining AI behavior boundaries
- ☐ DESIGN.md for visual consistency
- ☐ Feature specs before implementation
- ☐ E2E tests for critical user flows
- ☐ Code review process (even if reviewing your own PRs)
- ☐ Consistent commit messages and branch strategy

Stage 4 (Production):

- ☐ Automated tests run on every pull request

- [] Staging environment for testing before production
- [] Error monitoring (services like Sentry)
- [] Performance monitoring
- [] Database backups
- [] Incident response plan (what to do when things break)

Where to Go from Here

You've covered a lot of ground in this book. You understand how websites work, how to build interactive pages, how to deploy them, how backends and databases work, how to structure a real app, how to manage code with Git, how to add authentication, how to review and test your work, and how to move from improvisation to deliberate, spec-driven development.

That's a genuine skill set. Not the same skill set as a computer science graduate, but an increasingly valuable one. The ability to define what software should do, direct AI tools to build it, review the output critically, and iterate toward a quality product, that's a real capability that companies need.

Here are concrete next steps depending on what you want to build:

Internal tools for your company. Start with the problem that wastes the most time. Build a spec. Use the stack from this book (Next.js, PostgreSQL, Clerk, Railway). Deploy it for your team. Iterate based on their feedback. You don't need permission from IT for a proof of concept.

A product idea. Build the simplest possible version that tests your core assumption. One feature, done well, deployed to real users. Don't build the full vision. Build the smallest thing that proves the idea works, then grow from there.

Deeper technical skills. If you've caught the building bug and want to understand more, learn TypeScript (JavaScript with type safety), learn SQL (the language for talking to databases directly), or explore a CSS framework like Tailwind for more control over design.

AI tool proficiency. The AI development tools are getting better rapidly. Follow what Claude Code, Cursor, and similar tools are shipping. The gap between what you can build with AI assistance and what a traditional developer can build

is shrinking. Your competitive advantage is understanding the problem domain and the users, combined with the ability to direct AI tools effectively.

The most important thing you can do is build. Reading about software development is useful preparation, but the learning happens when you're in the code, dealing with real problems, deploying to real users, and fixing real bugs. Pick a project, open your editor, write a spec, and start building.

What You Now Know

You understand why vibe coding, while great for exploration, breaks down as projects grow, and you know that the fix is better specifications, not abandoning AI tools. You can write a feature spec that gives Claude Code enough detail to build what you actually want. You know how CLAUDE.md, AGENTS.md, and DESIGN.md work together to create consistency across your entire project. You understand the spec-driven workflow: write the spec, let AI implement, review against the spec, and iterate. You can assess your project's maturity level and know what practices to add at each stage. And you have a clear picture of where to go next, whether that's building internal tools, launching a product, or deepening your technical skills. You started this book wondering what a website actually is. You're ending it knowing how to build, deploy, and maintain real software. That's not a small thing.